ITECH 2004 ASSIGNMENT SQL DATABASE DESIGN AND IMPLEMENTATION

Mohammad Ashraful Islam

30430589

Table of Contents

Executive Summary

This report outlines the development and implementation of a spatially enabled database using PostgreSQL and PostGIS for the management and analysis of geographic data related to 100 tourist spots in Paris, France. The assignment involves creating, inserting, and querying spatial data, utilizing polygons to represent geographic areas (districts) and points to represent tourist attractions.

Key tasks include:

1. **Data Creation and Insertion**: Ten overlapping polygons, each representing a district in Paris, were created using geographic coordinates. Additionally, 100 tourist spots were inserted into the database, ensuring that some of the points fall within the polygons, as required.
2. **Spatial Queries**: SQL queries were developed to analyse spatial relationships, including finding which tourist spots are contained within specific districts, identifying overlaps between polygons, and calculating distances between points.
3. **Extended SQL Functionality**: Advanced SQL functionality, such as stored procedures and triggers, was implemented to automate tasks like validating new data entries and performing spatial operations. This includes a trigger to check if newly inserted points fall within specific bounding areas.
4. **Intersection Over Union (IoU)**: Spatial analysis was performed to calculate the overlap between polygons (districts), and results were visualized using KML format for integration with Google Earth.

The report demonstrates the effective use of PostGIS for handling spatial data, including data modelling, insertion, querying, and advanced spatial operations. This spatial database serves as a valuable tool for analysing tourist data and understanding the geographic distribution of attractions across Paris.

Scenario

The scenario involves managing tourist attractions in Paris, France. The goal is to create a spatial database that includes various districts (represented as polygons) and tourist attractions (represented as points). This database will allow users to query spatial relationships between attractions and districts to support tourism planning and management.

Task 1: Create and Insert Data

- **Creation of the polygons**

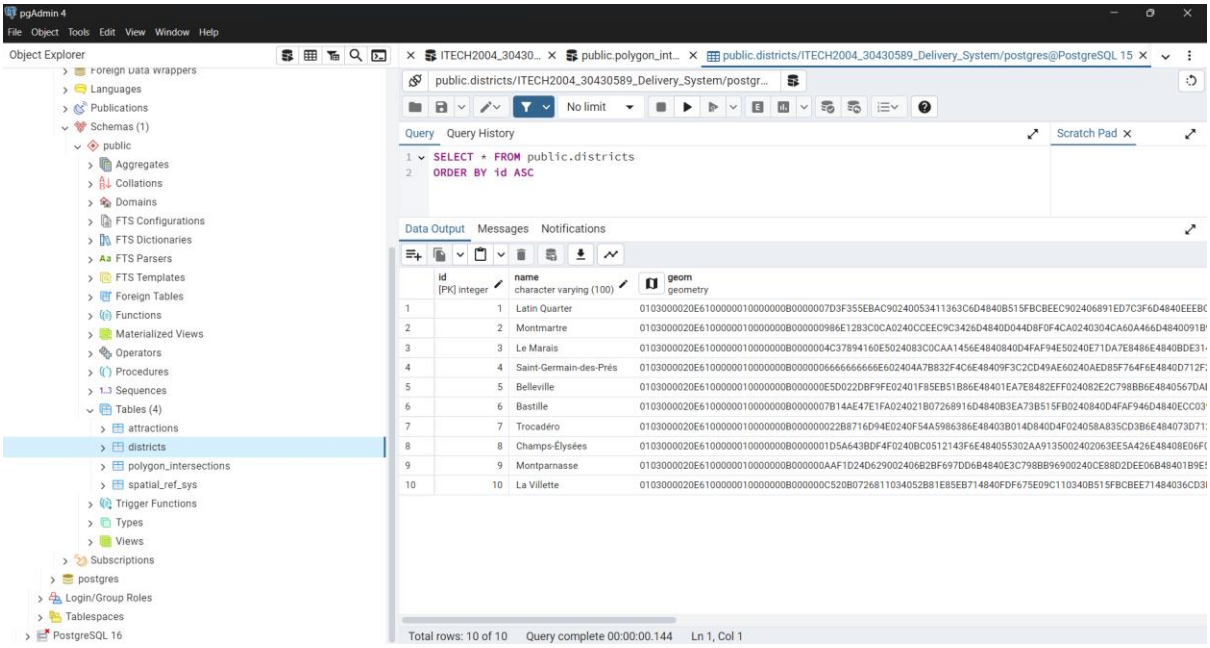
```
CREATE TABLE districts (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    geom GEOMETRY(POLYGON, 4326)
```

);

The geom column uses the GEOMETRY(POLYGON, 4326) type to store the spatial data for the district's boundaries as polygons, using the WGS 84 coordinate system (SRID 4326). This table is designed to store geographic data for different districts, including both names and their corresponding shapes on a map.

INSERT INTO districts (name, geom) VALUES ('Latin Quarter', ST_GeomFromText('POLYGON((2.3485 48.8534, 2.3486 48.8535, 2.3487 48.8536, 2.3488 48.8537, 2.3489 48.8538, 2.3490 48.8539, 2.3491 48.8540, 2.3492 48.8541, 2.3493 48.8542, 2.3494 48.8543, 2.3485 48.8534))', 4326)), ('Montmartre', ST_GeomFromText('POLYGON((2.3489 48.8537, 2.3491 48.8538, 2.3492 48.8539, 2.3493 48.8540, 2.3494 48.8541, 2.3495 48.8542, 2.3496 48.8543, 2.3497 48.8544, 2.3498 48.8545, 2.3489 48.8537))', 4326)), ('Belleville', ST_GeomFromText('POLYGON((2.3745 48.8650, 2.3746 48.8651, 2.3747 48.8652, 2.3748 48.8653, 2.3749 48.8654, 2.3750 48.8655, 2.3751 48.8656, 2.3752 48.8657, 2.3753 48.8658, 2.3745 48.8650))', 4326)), ('Le Marais', ST_GeomFromText('POLYGON((2.3745 48.8650, 2.3746 48.8651, 2.3747 48.8652, 2.3748 48.8653, 2.3749 48.8654, 2.3750 48.8655, 2.3751 48.8656, 2.3752 48.8657, 2.3753 48.8658, 2.3754 48.8659, 2.3745 48.8650))', 4326)), ('Saint-Germain-des-Prés', ST_GeomFromText('POLYGON((2.3625 48.8617, 2.3626 48.8618, 2.3627 48.8619, 2.3628 48.8620, 2.3629 48.8621, 2.3630 48.8622, 2.3631 48.8623, 2.3632 48.8624, 2.3633 48.8625, 2.3634 48.8626, 2.3625 48.8617))', 4326)), ('Bastille', ST_GeomFromText('POLYGON((2.3725 48.8560, 2.3726 48.8561, 2.3727 48.8562, 2.3728 48.8563, 2.3729 48.8564, 2.3730 48.8565, 2.3731 48.8566, 2.3732 48.8567, 2.3733 48.8568, 2.3734 48.8569, 2.3725 48.8560))', 4326)), ('Trocadéro', ST_GeomFromText('POLYGON((2.2885 48.8611, 2.2886 48.8612, 2.2887 48.8613, 2.2888 48.8614, 2.2889 48.8615, 2.2890 48.8616, 2.2891 48.8617, 2.2892 48.8618, 2.2893 48.8619, 2.2894 48.8620, 2.2885 48.8611))', 4326)), ('Champs-Élysées', ST_GeomFromText('POLYGON((2.2890 48.8613, 2.2891 48.8614, 2.2892 48.8615, 2.2893 48.8616, 2.2894 48.8617, 2.2895 48.8618, 2.2896 48.8619, 2.2897 48.8620, 2.2898 48.8621, 2.2899 48.8622, 2.2890 48.8613))', 4326)), ('Montparnasse', ST_GeomFromText('POLYGON((2.3205 48.8427, 2.3206 48.8428, 2.3207 48.8429, 2.3208 48.8430, 2.3209 48.8431, 2.3210 48.8432, 2.3211 48.8433, 2.3212 48.8434, 2.3213 48.8435, 2.3214 48.8436, 2.3205 48.8427))', 4326)), ('La Villette', ST_GeomFromText('POLYGON((2.3835 48.8900, 2.3836 48.8901, 2.3837 48.8902, 2.3838 48.8903, 2.3839 48.8904, 2.3840 48.8905, 2.3841 48.8906, 2.3842 48.8907, 2.3843 48.8908, 2.3844 48.8909, 2.3835 48.8900))', 4326));

This SQL script inserts 10 districts into the districts table. Each district has a **name** and a **polygon geometry** that represents its area on a map. The ST_GeomFromText function converts the polygon data from a text format (WKT) into a PostGIS-compatible geometry. Each polygon is defined by a series of latitude and longitude points (in pairs), outlining the district's boundaries. The SRID 4326 ensures the coordinates are based on the WGS 84 system, commonly used for GPS data. Districts like 'Latin Quarter', 'Montmartre', and 'Le Marais' are inserted with their corresponding polygons. The first and last coordinates in each polygon are typically the same to close the shape. This query allows the districts to be spatially managed in a PostGIS-enabled PostgreSQL database, enabling geographic operations and queries.



- **Creation of the points**

```
CREATE TABLE attractions (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    geom GEOMETRY(POINT, 4326)
);

INSERT INTO attractions (name, geom) VALUES ('Eiffel Tower',
ST_SetSRID(ST_MakePoint(2.2945, 48.8584), 4326));
INSERT INTO attractions (name, geom) VALUES ('Louvre Museum',
ST_SetSRID(ST_MakePoint(2.3376, 48.8606), 4326));
INSERT INTO attractions (name, geom) VALUES ('Notre-Dame Cathedral',
ST_SetSRID(ST_MakePoint(2.3499, 48.8530), 4326));
INSERT INTO attractions (name, geom) VALUES ('Arc de Triomphe',
ST_SetSRID(ST_MakePoint(2.2950, 48.8738), 4326));
INSERT INTO attractions (name, geom) VALUES ('Sacré-Cœur Basilica',
ST_SetSRID(ST_MakePoint(2.3431, 48.8867), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée d''Orsay',
ST_SetSRID(ST_MakePoint(2.3266, 48.8600), 4326));
INSERT INTO attractions (name, geom) VALUES ('Palais Garnier',
ST_SetSRID(ST_MakePoint(2.3318, 48.8704), 4326));
INSERT INTO attractions (name, geom) VALUES ('Luxembourg Gardens',
ST_SetSRID(ST_MakePoint(2.3371, 48.8462), 4326));
INSERT INTO attractions (name, geom) VALUES ('Place de la Concorde',
ST_SetSRID(ST_MakePoint(2.3212, 48.8656), 4326));
INSERT INTO attractions (name, geom) VALUES ('Centre Pompidou',
ST_SetSRID(ST_MakePoint(2.3522, 48.8606), 4326));
INSERT INTO attractions (name, geom) VALUES ('Pont Alexandre III',
ST_SetSRID(ST_MakePoint(2.3131, 48.8637), 4326));
```

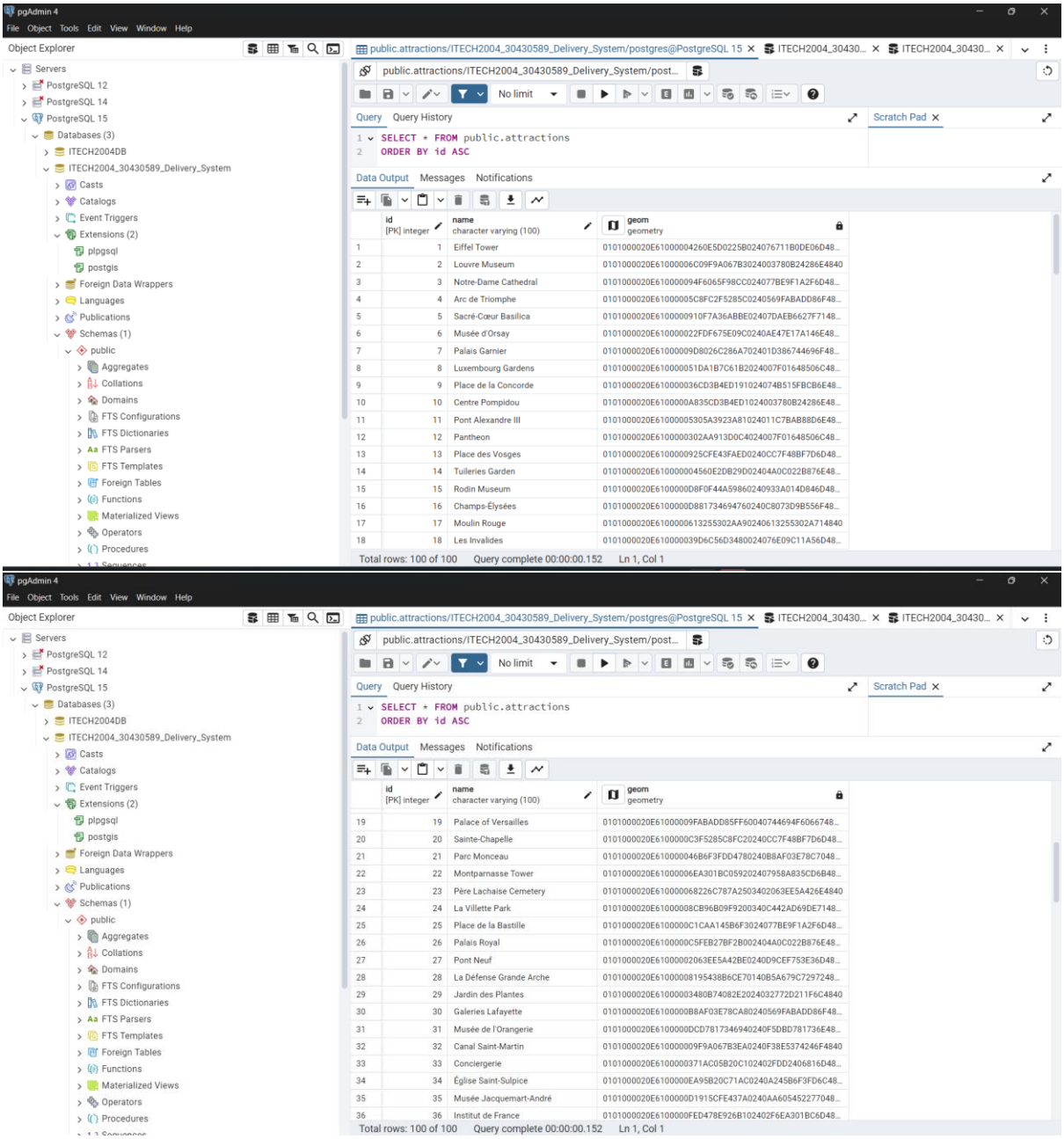
```
INSERT INTO attractions (name, geom) VALUES ('Pantheon',
ST_SetSRID(ST_MakePoint(2.3461, 48.8462), 4326));
INSERT INTO attractions (name, geom) VALUES ('Place des Vosges',
ST_SetSRID(ST_MakePoint(2.3662, 48.8554), 4326));
INSERT INTO attractions (name, geom) VALUES ('Tuileries Garden',
ST_SetSRID(ST_MakePoint(2.3270, 48.8635), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rodin Museum',
ST_SetSRID(ST_MakePoint(2.3156, 48.8556), 4326));
INSERT INTO attractions (name, geom) VALUES ('Champs-Élysées',
ST_SetSRID(ST_MakePoint(2.3079, 48.8698), 4326));
INSERT INTO attractions (name, geom) VALUES ('Moulin Rouge',
ST_SetSRID(ST_MakePoint(2.3326, 48.8841), 4326));
INSERT INTO attractions (name, geom) VALUES ('Les Invalides',
ST_SetSRID(ST_MakePoint(2.3126, 48.8566), 4326));
INSERT INTO attractions (name, geom) VALUES ('Palace of Versailles',
ST_SetSRID(ST_MakePoint(2.1203, 48.8049), 4326));
INSERT INTO attractions (name, geom) VALUES ('Sainte-Chapelle',
ST_SetSRID(ST_MakePoint(2.3450, 48.8554), 4326));
INSERT INTO attractions (name, geom) VALUES ('Parc Monceau',
ST_SetSRID(ST_MakePoint(2.3090, 48.8793), 4326));
INSERT INTO attractions (name, geom) VALUES ('Montparnasse Tower',
ST_SetSRID(ST_MakePoint(2.3213, 48.8422), 4326));
INSERT INTO attractions (name, geom) VALUES ('Père Lachaise Cemetery',
ST_SetSRID(ST_MakePoint(2.3933, 48.8614), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Villette Park',
ST_SetSRID(ST_MakePoint(2.3911, 48.8896), 4326));
INSERT INTO attractions (name, geom) VALUES ('Place de la Bastille',
ST_SetSRID(ST_MakePoint(2.3690, 48.8530), 4326));
INSERT INTO attractions (name, geom) VALUES ('Palais Royal',
ST_SetSRID(ST_MakePoint(2.3364, 48.8635), 4326));
INSERT INTO attractions (name, geom) VALUES ('Pont Neuf',
ST_SetSRID(ST_MakePoint(2.3429, 48.8585), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Défense Grande Arche',
ST_SetSRID(ST_MakePoint(2.2380, 48.8919), 4326));
INSERT INTO attractions (name, geom) VALUES ('Jardin des Plantes',
ST_SetSRID(ST_MakePoint(2.3606, 48.8447), 4326));
INSERT INTO attractions (name, geom) VALUES ('Galeries Lafayette',
ST_SetSRID(ST_MakePoint(2.3323, 48.8738), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée de l''Orangerie',
ST_SetSRID(ST_MakePoint(2.3224, 48.8629), 4326));
INSERT INTO attractions (name, geom) VALUES ('Canal Saint-Martin',
ST_SetSRID(ST_MakePoint(2.3646, 48.8683), 4326));
INSERT INTO attractions (name, geom) VALUES ('Conciergerie',
ST_SetSRID(ST_MakePoint(2.3443, 48.8555), 4326));
INSERT INTO attractions (name, geom) VALUES ('Église Saint-Sulpice',
ST_SetSRID(ST_MakePoint(2.3342, 48.8515), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée Jacquemart-André',
ST_SetSRID(ST_MakePoint(2.3097, 48.8762), 4326));
INSERT INTO attractions (name, geom) VALUES ('Institut de France',
ST_SetSRID(ST_MakePoint(2.3365, 48.8573), 4326));
```

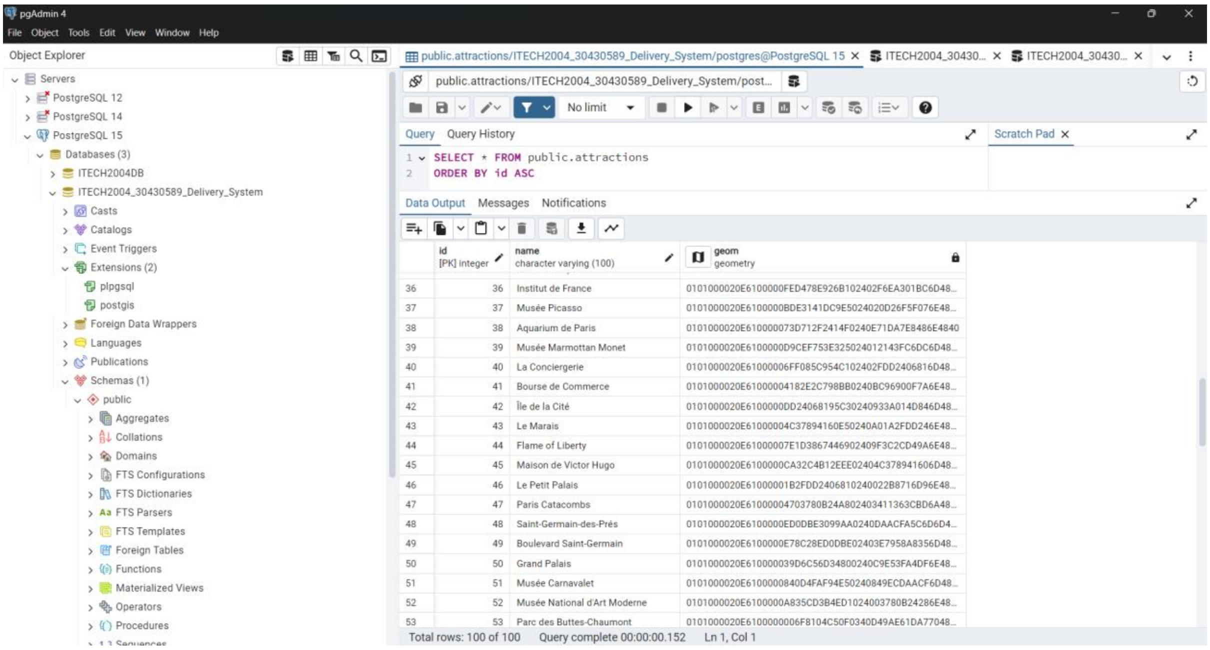
```
INSERT INTO attractions (name, geom) VALUES ('Musée Picasso',
ST_SetSRID(ST_MakePoint(2.3622, 48.8596), 4326));
INSERT INTO attractions (name, geom) VALUES ('Aquarium de Paris',
ST_SetSRID(ST_MakePoint(2.2887, 48.8616), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée Marmottan Monet',
ST_SetSRID(ST_MakePoint(2.2685, 48.8583), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Conciergerie',
ST_SetSRID(ST_MakePoint(2.3444, 48.8555), 4326));
INSERT INTO attractions (name, geom) VALUES ('Bourse de Commerce',
ST_SetSRID(ST_MakePoint(2.3416, 48.8631), 4326));
INSERT INTO attractions (name, geom) VALUES ('Île de la Cité',
ST_SetSRID(ST_MakePoint(2.3455, 48.8556), 4326));
INSERT INTO attractions (name, geom) VALUES ('Le Marais',
ST_SetSRID(ST_MakePoint(2.3620, 48.8605), 4326));
INSERT INTO attractions (name, geom) VALUES ('Flame of Liberty',
ST_SetSRID(ST_MakePoint(2.3014, 48.8641), 4326));
INSERT INTO attractions (name, geom) VALUES ('Maison de Victor Hugo',
ST_SetSRID(ST_MakePoint(2.3663, 48.8545), 4326));
INSERT INTO attractions (name, geom) VALUES ('Le Petit Palais',
ST_SetSRID(ST_MakePoint(2.3130, 48.8660), 4326));
INSERT INTO attractions (name, geom) VALUES ('Paris Catacombs',
ST_SetSRID(ST_MakePoint(2.3321, 48.8339), 4326));
INSERT INTO attractions (name, geom) VALUES ('Saint-Germain-des-Prés',
ST_SetSRID(ST_MakePoint(2.3333, 48.8549), 4326));
INSERT INTO attractions (name, geom) VALUES ('Boulevard Saint-Germain',
ST_SetSRID(ST_MakePoint(2.3428, 48.8532), 4326));
INSERT INTO attractions (name, geom) VALUES ('Grand Palais',
ST_SetSRID(ST_MakePoint(2.3126, 48.8662), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée Carnavalet',
ST_SetSRID(ST_MakePoint(2.3621, 48.8579), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée National d''Art Moderne',
ST_SetSRID(ST_MakePoint(2.3522, 48.8606), 4326));
INSERT INTO attractions (name, geom) VALUES ('Parc des Buttes-Chaumont',
ST_SetSRID(ST_MakePoint(2.3827, 48.8801), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue de Rivoli',
ST_SetSRID(ST_MakePoint(2.3427, 48.8592), 4326));
INSERT INTO attractions (name, geom) VALUES ('Bois de Boulogne',
ST_SetSRID(ST_MakePoint(2.2449, 48.8687), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Grande Mosquée de Paris',
ST_SetSRID(ST_MakePoint(2.3551, 48.8416), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue Mouffetard',
ST_SetSRID(ST_MakePoint(2.3492, 48.8414), 4326));
INSERT INTO attractions (name, geom) VALUES ('Paris Zoological Park',
ST_SetSRID(ST_MakePoint(2.4164, 48.8345), 4326));
INSERT INTO attractions (name, geom) VALUES ('Parc André Citroën',
ST_SetSRID(ST_MakePoint(2.2766, 48.8402), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue Montorgueil',
ST_SetSRID(ST_MakePoint(2.3481, 48.8657), 4326));
INSERT INTO attractions (name, geom) VALUES ('Marché aux Fleurs Reine Elizabeth II',
ST_SetSRID(ST_MakePoint(2.3470, 48.8563), 4326));
```

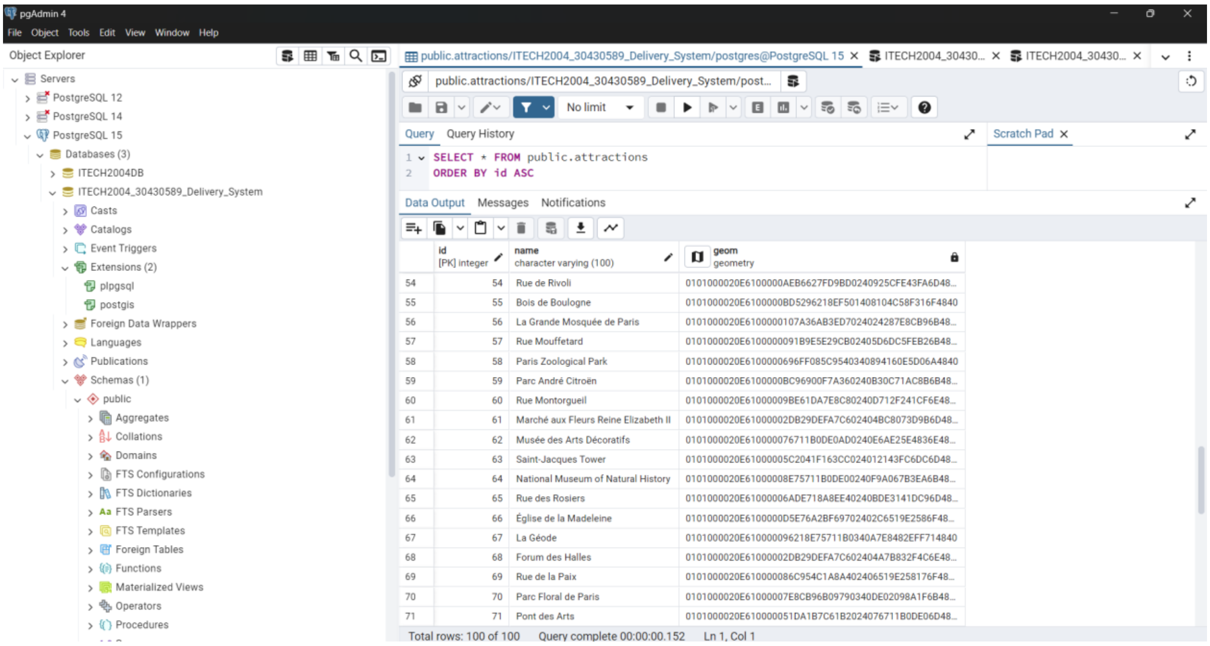
```
INSERT INTO attractions (name, geom) VALUES ('Musée des Arts Décoratifs',
ST_SetSRID(ST_MakePoint(2.3349, 48.8634), 4326));
INSERT INTO attractions (name, geom) VALUES ('Saint-Jacques Tower',
ST_SetSRID(ST_MakePoint(2.3498, 48.8583), 4326));
INSERT INTO attractions (name, geom) VALUES ('National Museum of Natural History',
ST_SetSRID(ST_MakePoint(2.3594, 48.8431), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue des Rosiers',
ST_SetSRID(ST_MakePoint(2.3616, 48.8577), 4326));
INSERT INTO attractions (name, geom) VALUES ('Église de la Madeleine',
ST_SetSRID(ST_MakePoint(2.3242, 48.8699), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Géode',
ST_SetSRID(ST_MakePoint(2.3884, 48.8906), 4326));
INSERT INTO attractions (name, geom) VALUES ('Forum des Halles',
ST_SetSRID(ST_MakePoint(2.3470, 48.8617), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue de la Paix',
ST_SetSRID(ST_MakePoint(2.3304, 48.8679), 4326));
INSERT INTO attractions (name, geom) VALUES ('Parc Floral de Paris',
ST_SetSRID(ST_MakePoint(2.4341, 48.8369), 4326));
INSERT INTO attractions (name, geom) VALUES ('Pont des Arts',
ST_SetSRID(ST_MakePoint(2.3371, 48.8584), 4326));
INSERT INTO attractions (name, geom) VALUES ('Colonne Vendôme',
ST_SetSRID(ST_MakePoint(2.3282, 48.8676), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue du Faubourg Saint-Honoré',
ST_SetSRID(ST_MakePoint(2.3221, 48.8690), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée du Quai Branly',
ST_SetSRID(ST_MakePoint(2.2974, 48.8603), 4326));
INSERT INTO attractions (name, geom) VALUES ('Palais de Tokyo',
ST_SetSRID(ST_MakePoint(2.2960, 48.8650), 4326));
INSERT INTO attractions (name, geom) VALUES ('Grande Galerie de l''Évolution',
ST_SetSRID(ST_MakePoint(2.3561, 48.8435), 4326));
INSERT INTO attractions (name, geom) VALUES ('Place Vendôme',
ST_SetSRID(ST_MakePoint(2.3284, 48.8674), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue des Martyrs',
ST_SetSRID(ST_MakePoint(2.3426, 48.8791), 4326));
INSERT INTO attractions (name, geom) VALUES ('Montparnasse Cemetery',
ST_SetSRID(ST_MakePoint(2.3267, 48.8381), 4326));
INSERT INTO attractions (name, geom) VALUES ('Le Bon Marché',
ST_SetSRID(ST_MakePoint(2.3244, 48.8501), 4326));
INSERT INTO attractions (name, geom) VALUES ('Marché d''Aligre',
ST_SetSRID(ST_MakePoint(2.3802, 48.8495), 4326));
INSERT INTO attractions (name, geom) VALUES ('Musée Guimet',
ST_SetSRID(ST_MakePoint(2.2937, 48.8641), 4326));
INSERT INTO attractions (name, geom) VALUES ('Avenue Foch',
ST_SetSRID(ST_MakePoint(2.2829, 48.8728), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue du Bac',
ST_SetSRID(ST_MakePoint(2.3267, 48.8543), 4326));
INSERT INTO attractions (name, geom) VALUES ('Pavillon de l''Arsenal',
ST_SetSRID(ST_MakePoint(2.3666, 48.8539), 4326));
INSERT INTO attractions (name, geom) VALUES ('Parc Montsouris',
ST_SetSRID(ST_MakePoint(2.3405, 48.8217), 4326));
```

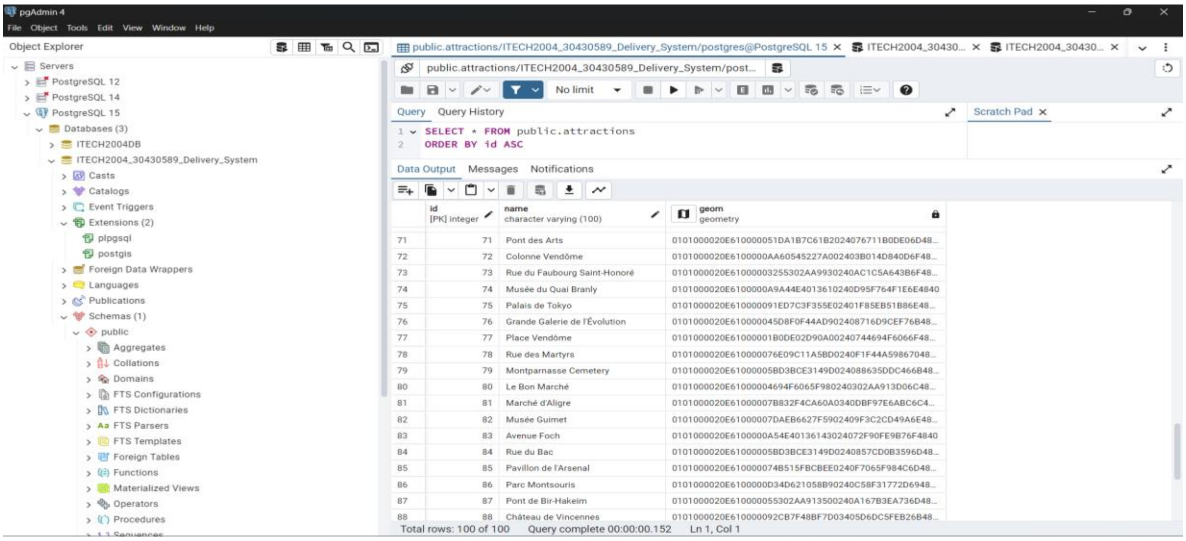
```
INSERT INTO attractions (name, geom) VALUES ('Pont de Bir-Hakeim',
ST_SetSRID(ST_MakePoint(2.2891, 48.8551), 4326));
INSERT INTO attractions (name, geom) VALUES ('Château de Vincennes',
ST_SetSRID(ST_MakePoint(2.4364, 48.8414), 4326));
INSERT INTO attractions (name, geom) VALUES ('Palais de Chaillot',
ST_SetSRID(ST_MakePoint(2.2882, 48.8625), 4326));
INSERT INTO attractions (name, geom) VALUES ('Gare Saint-Lazare',
ST_SetSRID(ST_MakePoint(2.3276, 48.8760), 4326));
INSERT INTO attractions (name, geom) VALUES ('Gare de Lyon',
ST_SetSRID(ST_MakePoint(2.3730, 48.8443), 4326));
INSERT INTO attractions (name, geom) VALUES ('Place d''Italie',
ST_SetSRID(ST_MakePoint(2.3556, 48.8327), 4326));
INSERT INTO attractions (name, geom) VALUES ('École Militaire',
ST_SetSRID(ST_MakePoint(2.3030, 48.8537), 4326));
INSERT INTO attractions (name, geom) VALUES ('Rue Oberkampf',
ST_SetSRID(ST_MakePoint(2.3766, 48.8646), 4326));
INSERT INTO attractions (name, geom) VALUES ('Square du Vert-Galant',
ST_SetSRID(ST_MakePoint(2.3400, 48.8572), 4326));
INSERT INTO attractions (name, geom) VALUES ('Gare du Nord',
ST_SetSRID(ST_MakePoint(2.3553, 48.8809), 4326));
INSERT INTO attractions (name, geom) VALUES ('Église Saint-Eustache',
ST_SetSRID(ST_MakePoint(2.3450, 48.8626), 4326));
INSERT INTO attractions (name, geom) VALUES ('La Sorbonne University',
ST_SetSRID(ST_MakePoint(2.3435, 48.8487), 4326));
INSERT INTO attractions (name, geom) VALUES ('Galerie Vivienne',
ST_SetSRID(ST_MakePoint(2.3416, 48.8669), 4326));
INSERT INTO attractions (name, geom) VALUES ('Canal de l''Ourcq',
ST_SetSRID(ST_MakePoint(2.3906, 48.8922), 4326));
```

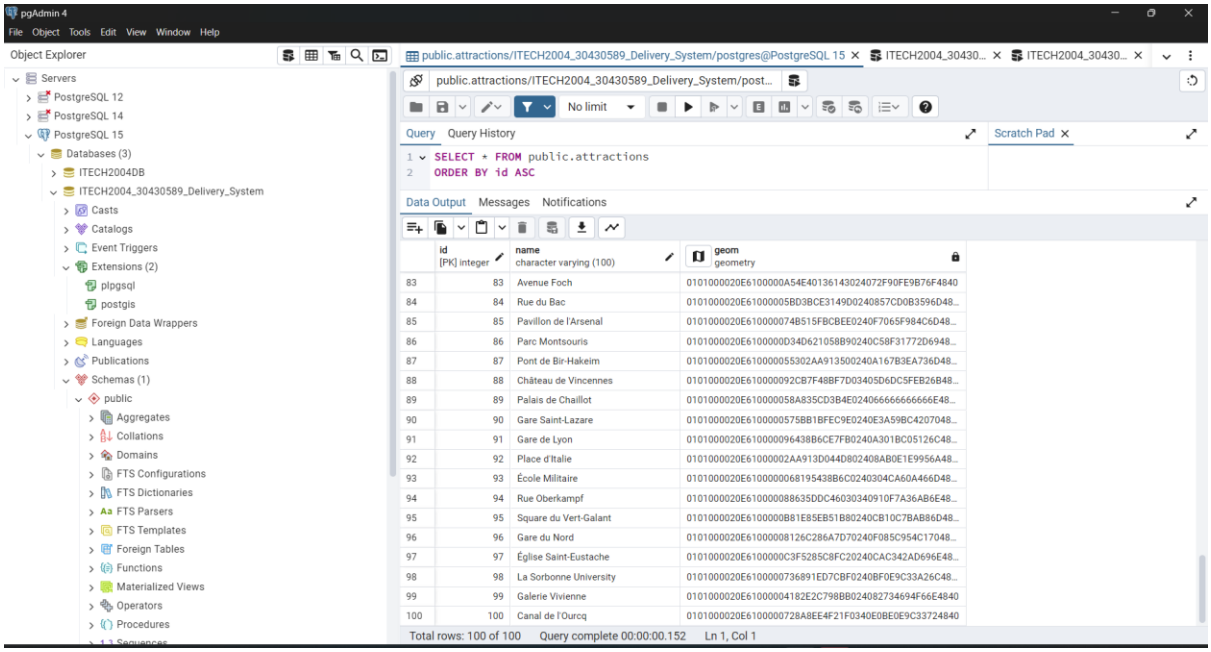
This SQL script inserts 100 tourist attractions into the attractions table. Each attraction is represented by a name and a geographic location stored as a **point** using ST_SetSRID(ST_MakePoint(...)), which sets the coordinates in **latitude** and **longitude** format using the WGS 84 coordinate system (SRID 4326). The attractions include notable landmarks like the **Eiffel Tower**, **Louvre Museum**, **Notre-Dame Cathedral**, and many more across Paris. The geographic data is crucial for spatial queries and geographic operations, allowing the system to accurately plot these attractions on a map.


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers
PostgreSQL 12
PostgreSQL 14
PostgreSQL 15
Databases (3)
ITECH2004DB
ITECH2004_30430589_Delivery_System
Casts
Catalogs
Event Triggers
Extensions (2)
plpgsql
postgis
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
public.attractions/ITECH2004_30430589_Delivery_System/postgres@PostgreSQL 15
public.attractions/ITECH2004_30430589_Delivery_System/post...
No limit
Query
Query History
Scratch Pad
SELECT * FROM public.attractions
ORDER BY id ASC
Data Output
Messages
Notifications
id [PK] integer
name character varying (100)
geom geometry
1 1 Eiffel Tower 0101000020E6100000...
2 2 Louvre Museum 0101000020E6100000...
3 3 Notre-Dame Cathedral 0101000020E6100000...
4 4 Arc de Triomphe 0101000020E6100000...
5 5 Sacré-Cœur Basilica 0101000020E6100000...
6 6 Musée d'Orsay 0101000020E6100000...
7 7 Palais Garnier 0101000020E6100000...
8 8 Luxembourg Gardens 0101000020E6100000...
9 9 Place de la Concorde 0101000020E6100000...
10 10 Centre Pompidou 0101000020E6100000...
11 11 Pont Alexandre III 0101000020E6100000...
12 12 Pantheon 0101000020E6100000...
13 13 Place des Vosges 0101000020E6100000...
14 14 Tuileries Garden 0101000020E6100000...
15 15 Rodin Museum 0101000020E6100000...
16 16 Champs-Élysées 0101000020E6100000...
17 17 Moulin Rouge 0101000020E6100000...
18 18 Les Invalides 0101000020E6100000...
Total rows: 100 of 100 Query complete 00:00:00.152 Ln 1, Col 1
19 19 Palace of Versailles 0101000020E6100000...
20 20 Sainte-Chapelle 0101000020E6100000...
21 21 Parc Monceau 0101000020E6100000...
22 22 Montparnasse Tower 0101000020E6100000...
23 23 Père Lachaise Cemetery 0101000020E6100000...
24 24 La Villette Park 0101000020E6100000...
25 25 Place de la Bastille 0101000020E6100000...
26 26 Palais Royal 0101000020E6100000...
27 27 Pont Neuf 0101000020E6100000...
28 28 La Défense Grande Arche 0101000020E6100000...
29 29 Jardin des Plantes 0101000020E6100000...
30 30 Galeries Lafayette 0101000020E6100000...
31 31 Musée de l'Orangerie 0101000020E6100000...
32 32 Canal Saint-Martin 0101000020E6100000...
33 33 Conciergerie 0101000020E6100000...
34 34 Église Saint-Sulpice 0101000020E6100000...
35 35 Musée Jacquemart-André 0101000020E6100000...
36 36 Institut de France 0101000020E6100000...
Total rows: 100 of 100 Query complete 00:00:00.152 Ln 1, Col 1


pgAdmin 4
Object Explorer
public.attractions/ITECH2004_30430589_Delivery_System/postgres@PostgreSQL 15
SELECT * FROM public.attractions
ORDER BY id ASC
Data Output
id [PK] integer
name character varying (100)
geom geometry
36 Institut de France
37 Musée Picasso
38 Aquarium de Paris
39 Musée Marmottan Monet
40 La Conciergerie
41 Bourse de Commerce
42 Île de la Cité
43 Le Marais
44 Flame of Liberty
45 Maison de Victor Hugo
46 Le Petit Palais
47 Paris Catacombs
48 Saint-Germain-des-Prés
49 Boulevard Saint-Germain
50 Grand Palais
51 Musée Carnavalet
52 Musée National d'Art Moderne
53 Parc des Buttes-Chaumont
Total rows: 100 of 100 Query complete 00:00:00.152 Ln 1, Col 1


pgAdmin 4
Object Explorer
public.attractions/ITECH2004_30430589_Delivery_System/postgres@PostgreSQL 15
SELECT * FROM public.attractions
ORDER BY id ASC
Data Output
id [PK] integer
name character varying (100)
geom geometry
54 Rue de Rivoli
55 Bois de Boulogne
56 La Grande Mosquée de Paris
57 Rue Mouffetard
58 Paris Zoological Park
59 Parc André Citroën
60 Rue Montorgueil
61 Marché aux Fleurs Reine Elizabeth II
62 Musée des Arts Décoratifs
63 Saint-Jacques Tower
64 National Museum of Natural History
65 Rue des Rosiers
66 Église de la Madeleine
67 La Géode
68 Forum des Halles
69 Rue de la Paix
70 Parc Floral de Paris
71 Pont des Arts
Total rows: 100 of 100 Query complete 00:00:00.152 Ln 1, Col 1


pgAdmin 4
Object Explorer
public.attractions/ITECH2004_30430589_Delivery_System/postgres@PostgreSQL 15
SELECT * FROM public.attractions
ORDER BY id ASC
Data Output
id [PK] integer
name character varying (100)
geom geometry
71 Pont des Arts
72 Colonne Vendôme
73 Rue du Faubourg Saint-Honoré
74 Musée du Quai Branly
75 Palais de Tokyo
76 Grande Galerie de l'Évolution
77 Place Vendôme
78 Rue des Martyrs
79 Montparnasse Cemetery
80 Le Bon Marché
81 Marché d'Aligre
82 Musée Guimet
83 Avenue Foch
84 Rue du Bac
85 Pavillon de l'Arsenal
86 Parc Montsouris
87 Pont de Bir-Hakeim
88 Château de Vincennes
Total rows: 100 of 100 Query complete 00:00:00.152 Ln 1, Col 1



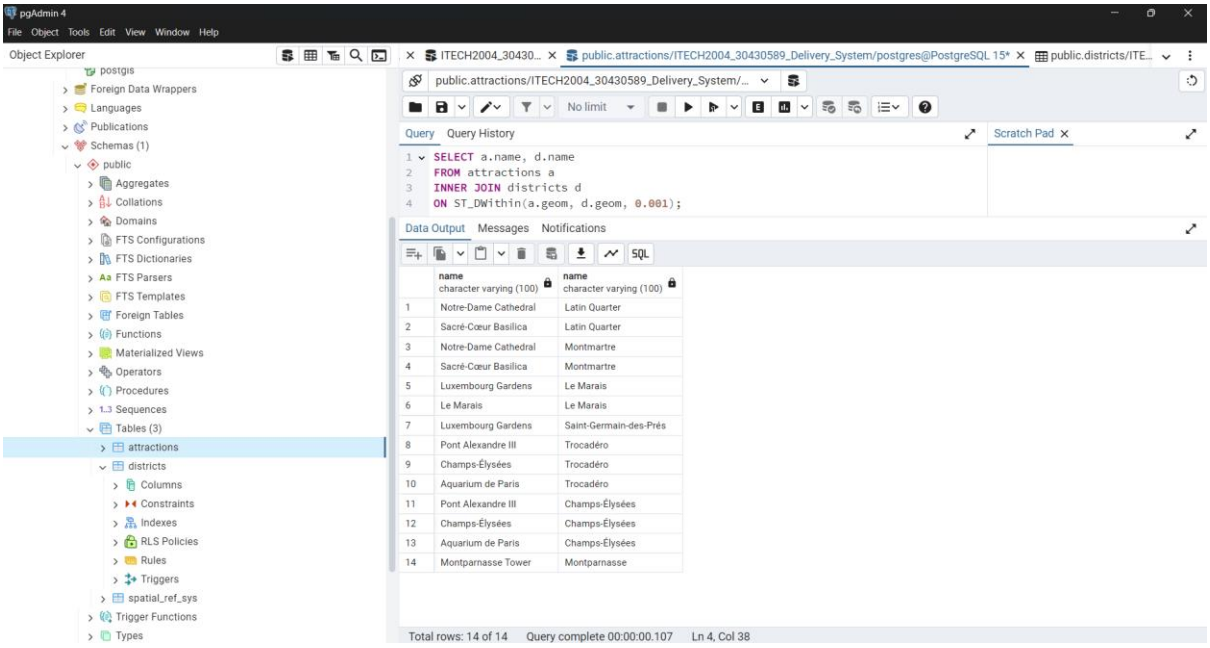
Task 2: SQL Queries

- **Difference Between an Inner Join and Outer Join**

  Inner Join example

  SELECT a.name, d.name
  FROM attractions a
  INNER JOIN districts d
  ON ST_DWithin(a.geom, d.geom, 0.001);

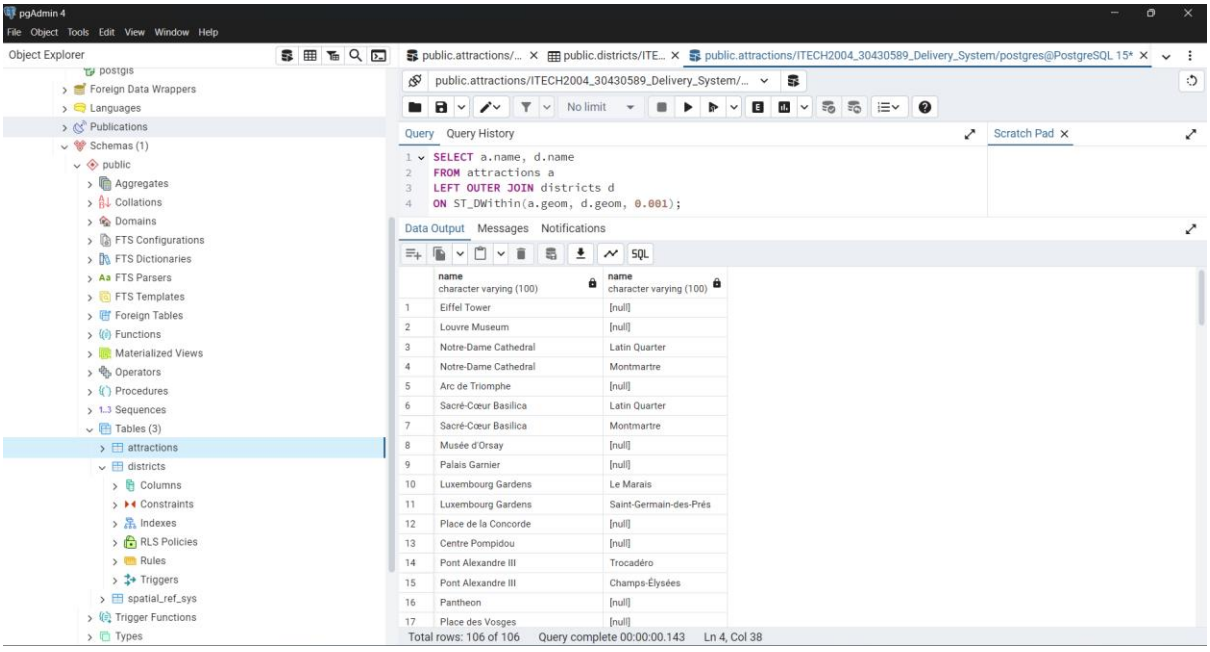
  Using the ST_DWithin() spatial function, this SQL query chooses the names of locations and attractions whose geometries are close to one another. The condition of the query determines whether the geometry of an attraction is within 0.001 degrees of the geometry of a district and executes an inner join between the attractions and districts tables. In order to account for tiny positional variations or rounding mistakes in the geographic coordinates, a tolerance of 0.001 is used. This allows the query to capture attractions that are either inside or very close to a district's boundary, even if their coordinates are not an exact match. In geographic terms, 0.001 degrees roughly translates to about 111 meters, making this buffer useful for ensuring attractions are correctly associated with nearby districts. By using this approach, the query avoids missing valid spatial relationships due to floating-point precision issues, which are common in geographic data. The result is a more flexible and accurate spatial join, returning attractions and districts that are geographically near each other.



Outer Join example

SELECT a.name, d.name
FROM attractions a
LEFT OUTER JOIN districts d
ON ST_DWithin(a.geom, d.geom, 0.001);

A left outer join is carried out by this query. It returns all attractions; however, if an attraction is not part of a district, the district name will still display with NULL values. Even in the event that there isn't a matching district, the outer join guarantees that you receive every record from the attractions table.



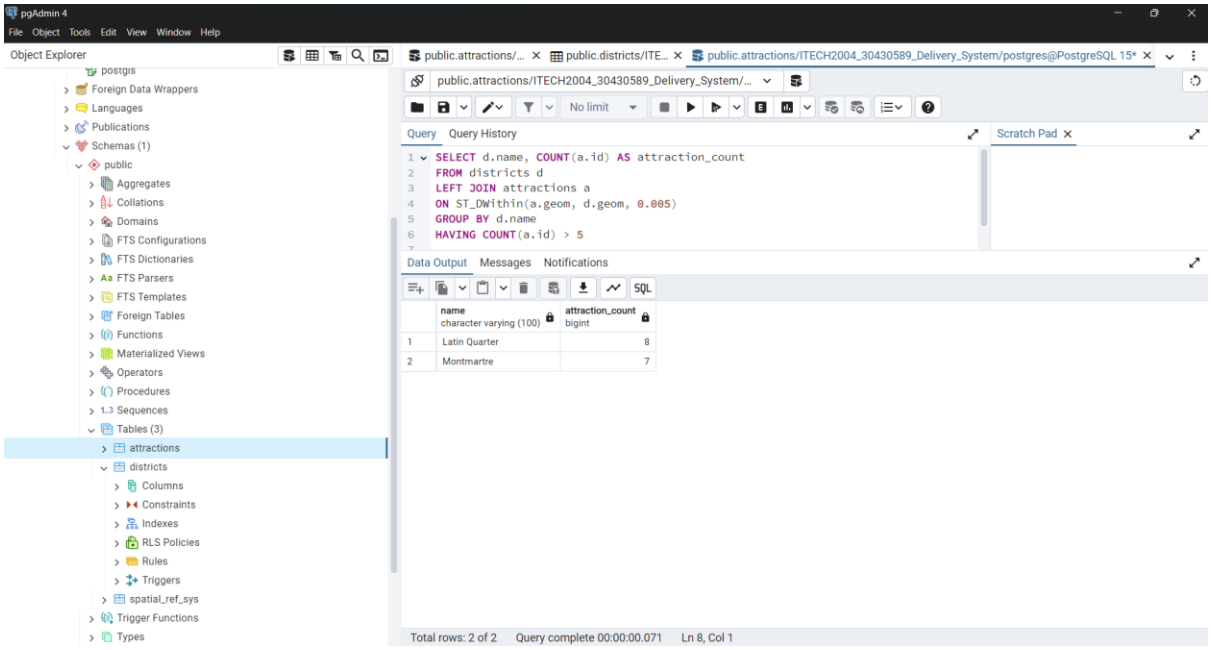
Purpose

- Inner Join: Only show attractions that are within a district.
- Outer Join: Show all attractions, even those not assigned to a district.

- **GROUP BY and HAVING Example**

```
SELECT d.name, COUNT(a.id) AS attraction_count
FROM districts d
LEFT JOIN attractions a
ON ST_DWithin(a.geom, d.geom, 0.005)
GROUP BY d.name
HAVING COUNT(a.id) > 5
```

The purpose of this query is to find out how many attractions are located within a certain distance (0.005 degrees, roughly equivalent to 500 meters) from each district in the database and filter out districts that have more than 5 nearby attractions.
The query uses a **LEFT JOIN** to ensure that all districts are included in the result set, even if they do not have any attractions nearby. The ST_DWithin function checks if each attraction is within the specified distance (0.005) of the district's boundary. It then groups the result by the name of the district and counts how many attractions fall within that distance. Finally, the HAVING clause filters the results, only showing districts that have more than 5 attractions within this range. This helps analyze the density of tourist attractions near each district, which could be useful for tourism, city planning, or marketing insights.



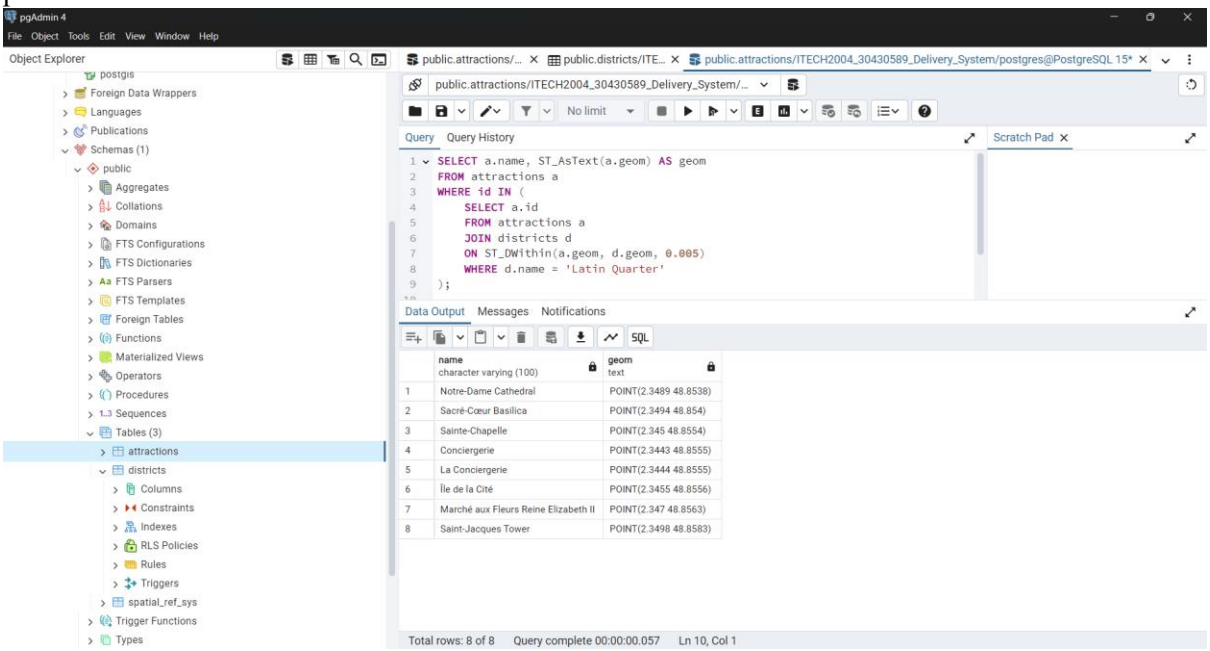
- **SQL Query Using Inner SQL Query**

```
SELECT a.name, ST_AsText(a.geom) AS geom
FROM attractions a
```

```
WHERE id IN (
   SELECT a.id
   FROM attractions a
   JOIN districts d
   ON ST_DWithin(a.geom, d.geom, 0.005)
   WHERE d.name = 'Latin Quarter'
);
```

The purpose of this query is to retrieve a list of attractions located within a certain proximity (0.005 units, likely degrees of latitude and longitude) to the district named "Latin Quarter" and display their names along with their geographical coordinates in a human-readable format. The query first uses a subquery to find all attractions that are within the defined distance from the "Latin Quarter" district using the ST_DWithin() spatial function. It then displays the name and coordinates (in Well-Known Text format) of these attractions.

For the database solution, this query is helpful in analyzing and visualizing spatial relationships between different districts and attractions in Paris. It allows the user to focus on attractions near a specific district and retrieve their exact coordinates, which is valuable for tasks such as mapping, location-based recommendations, or distance-based filtering of nearby places of interest.

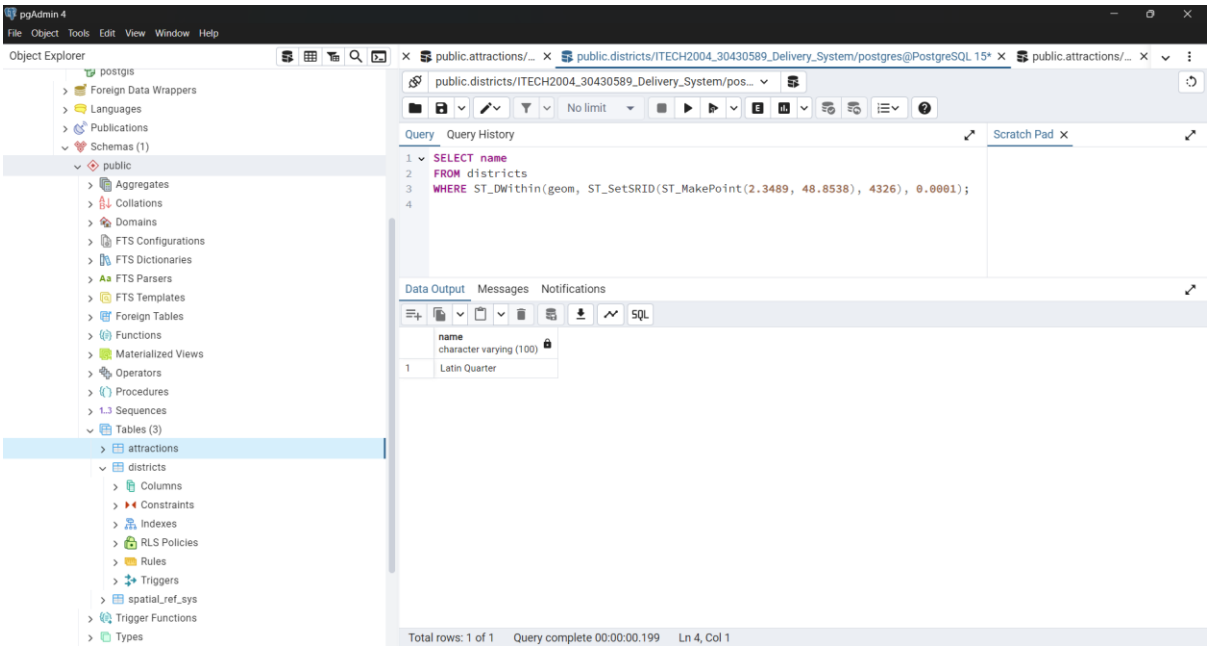


Task 3: Spatial SQL Extension

- **Query to find all polygons containing a given point**

```
SELECT name
FROM districts
WHERE ST_DWithin(geom, ST_SetSRID(ST_MakePoint(2.3489, 48.8538), 4326), 0.0001);
```

This query checks if a given point (in this case, at coordinates (2.3489, 48.8538)) is located within or very close to any polygons stored in the districts table. The query uses the ST_DWithin function, which returns true if the point is within a specified distance (here, 0.0001 degrees) from the boundary of a polygon. The purpose of the query in this database solution is to find the name of the district that contains the point or is very close to it, with a small tolerance for precision issues. This is useful in scenarios where exact boundaries may be difficult to work with due to minute rounding errors in geographic data. By using a small buffer, the query can ensure that points near the edges of polygons are correctly identified as being within those polygons. The result will be a list of district names where the point is either contained or within a close proximity to their boundaries.



- **Query to find the nearest polygon if the point is not within any polygon**

SELECT name, ST_Distance(geom, ST_SetSRID(ST_MakePoint(2.2945, 48.8584), 4326)) AS distance
FROM districts
ORDER BY distance
LIMIT 1;
The purpose of this query is to find the nearest district to a specific geographic point, in this case, the coordinates (2.2945, 48.8584), which corresponds to the Eiffel Tower. The query uses the ST_Distance function to calculate the distance between the point and the geometry (polygon) of each district in the districts table. It orders the results by the smallest distance first using the ORDER BY distance clause and limits the output to the closest match with LIMIT 1. This type of query is useful for spatial analysis in geographical databases. For example, in a tourism or location-based application, it can be used to determine which administrative area (district) is closest to a point of interest (such as a landmark or tourist spot), helping with proximity-based recommendations or categorization of points within certain boundaries.

Task 4: Extended SQL functionality

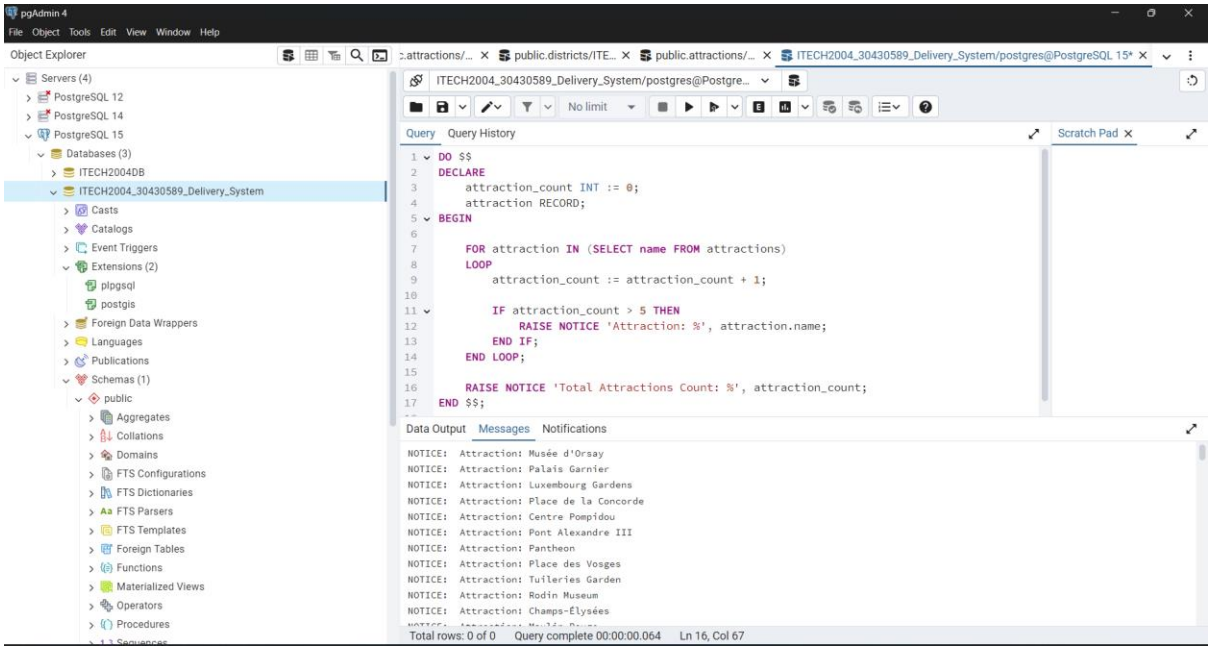
- **Running PL/SQL Blocks (Sequences, Selection, and Iteration)**

```
DO $$
DECLARE
   attraction_count INT := 0;
   attraction RECORD;
BEGIN
   FOR attraction IN (SELECT name FROM attractions)
   LOOP
      attraction_count := attraction_count + 1;

      IF attraction_count > 5 THEN
         RAISE NOTICE 'Attraction: %', attraction.name;
      END IF;
   END LOOP;

   RAISE NOTICE 'Total Attractions Count: %', attraction_count;
END $$;
```

The purpose of this PL/SQL block is to demonstrate sequence, selection, and iteration constructs while working with a set of data in a PostgreSQL database, specifically focusing on the *attractions* table.

- **Sequence**: It initializes the attraction_count variable to track the number of rows (attractions) processed.
- **Iteration**: The block iterates through each attraction in the *attractions* table using a FOR loop, incrementing the attraction_count on each iteration.
- **Selection**: It includes an IF condition to check if the attraction_count exceeds 5. Once this condition is met, the block prints the name of each attraction starting from the 6th one.
- **Notification**: After iterating through all the rows, the block raises a notice to print the total count of attractions, providing feedback on how many records were processed.

This block can be useful for monitoring large datasets in a database, checking specific conditions (in this case, checking when more than 5 attractions have been processed), and displaying feedback to the user without altering the underlying data.



- **Writing a Stored Procedure**

CREATE OR REPLACE PROCEDURE find_district_for_point(x_coordinate FLOAT, y_coordinate FLOAT)

LANGUAGE plpgsql

AS $$

DECLARE

district_name TEXT;

BEGIN

SELECT name INTO district_name

FROM districts

WHERE ST_Contains(geom, ST_SetSRID(ST_MakePoint(x_coordinate, y_coordinate), 4326));

IF district_name IS NULL THEN

RAISE NOTICE 'No district found for the given point';

ELSE

RAISE NOTICE 'The point is within the district: %', district_name;

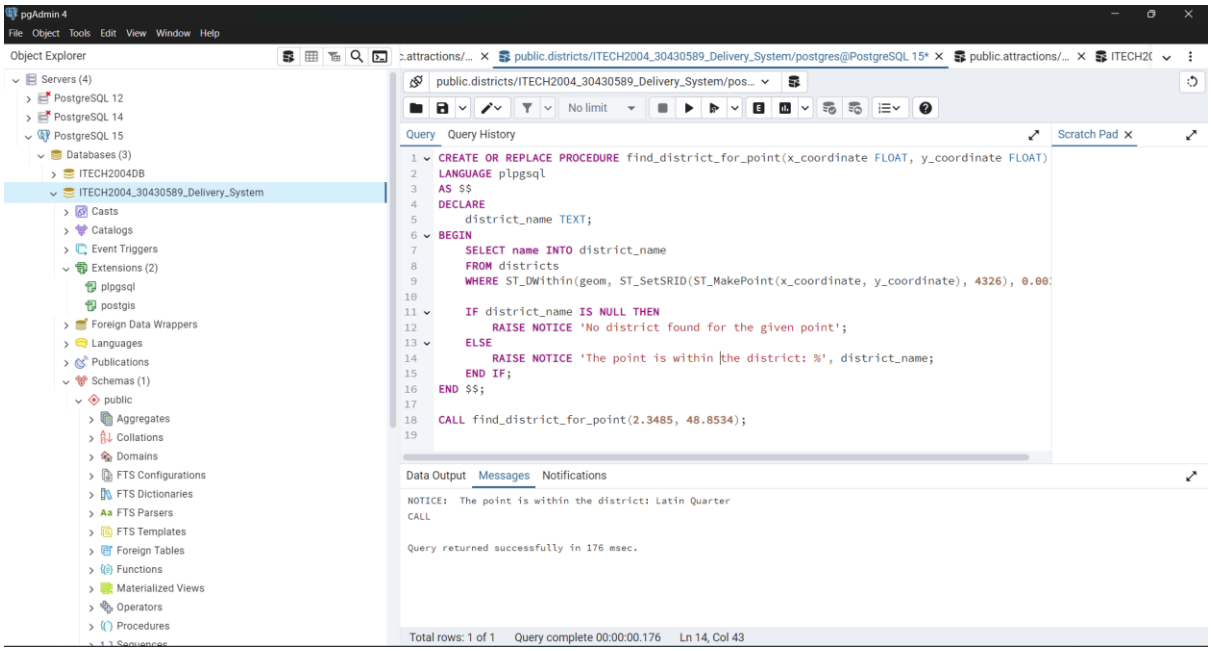
END IF;

END;

$$;

The stored procedure checks if a given point (defined by x_coordinate and y_coordinate) lies within any district. It uses the ST_Contains function to check if the point is inside a polygon

representing a district in the districts table. If a district is found containing the point, the procedure raises a notice displaying the district's name. If no district contains the point, it raises a notice indicating that no district was found. This procedure is useful for identifying which geographic area a specific point belongs to, commonly applied in mapping or spatial analysis tasks.



- **After Insert Trigger**

Creating Trigger Function

```
CREATE OR REPLACE FUNCTION check_point_within_bounding_box()
RETURNS TRIGGER AS $$
DECLARE
   bounding_box GEOMETRY := ST_SetSRID(
      ST_MakePolygon(
         ST_GeomFromText('LINESTRING(2.2900 48.8500, 2.2900 48.8600, 2.3000
48.8600, 2.3000 48.8500, 2.2900 48.8500)')
      ), 4326);
   point_inside BOOLEAN;
BEGIN

   point_inside := ST_Contains(bounding_box, NEW.geom);

   IF point_inside THEN
      RAISE NOTICE 'The point is within the bounding box';
   ELSE
      RAISE NOTICE 'The point is outside the bounding box';
   END IF;

   RETURN NEW;
END;
```

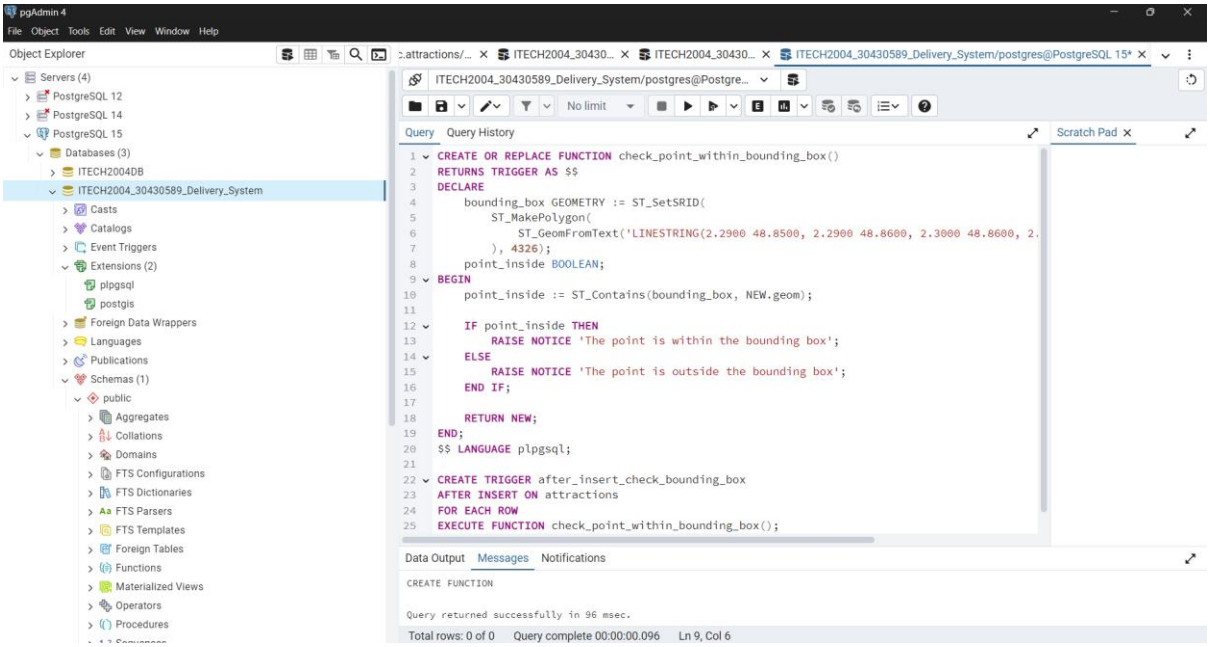
$$ LANGUAGE plpgsql;

Creating Trigger

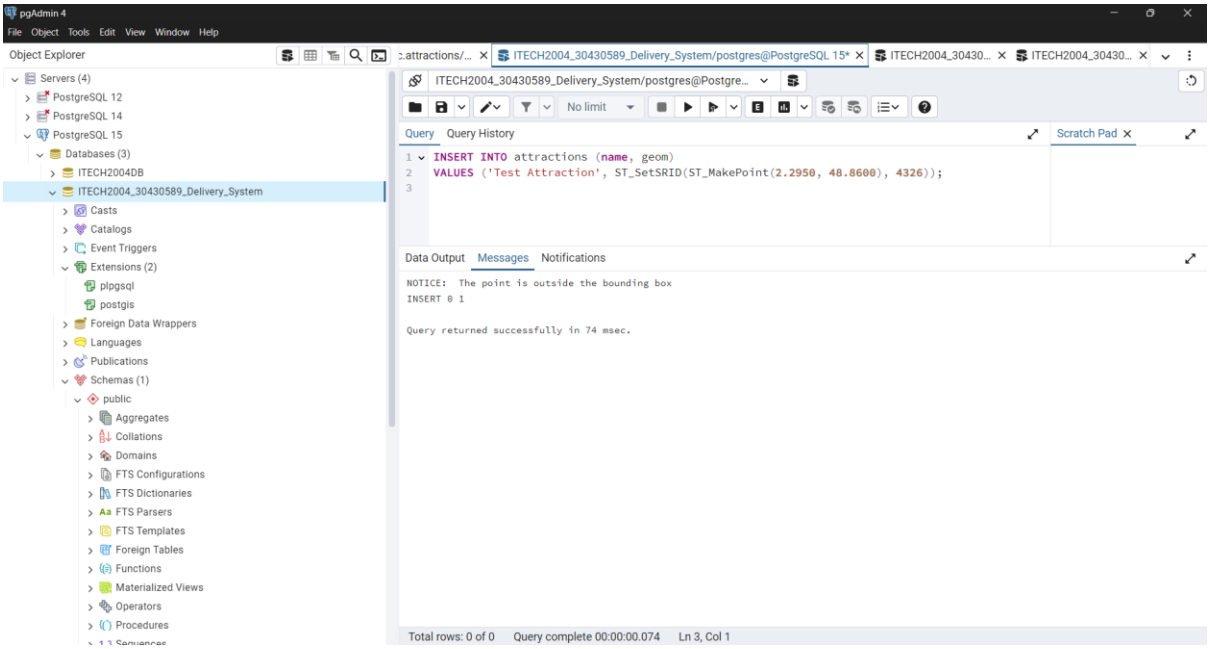
CREATE TRIGGER after_insert_check_bounding_box
AFTER INSERT ON attractions
FOR EACH ROW
EXECUTE FUNCTION check_point_within_bounding_box();

Testing Trigger

INSERT INTO attractions (name, geom)
VALUES ('Test Attraction', ST_SetSRID(ST_MakePoint(2.2950, 48.8600), 4326));



Testing Trigger:



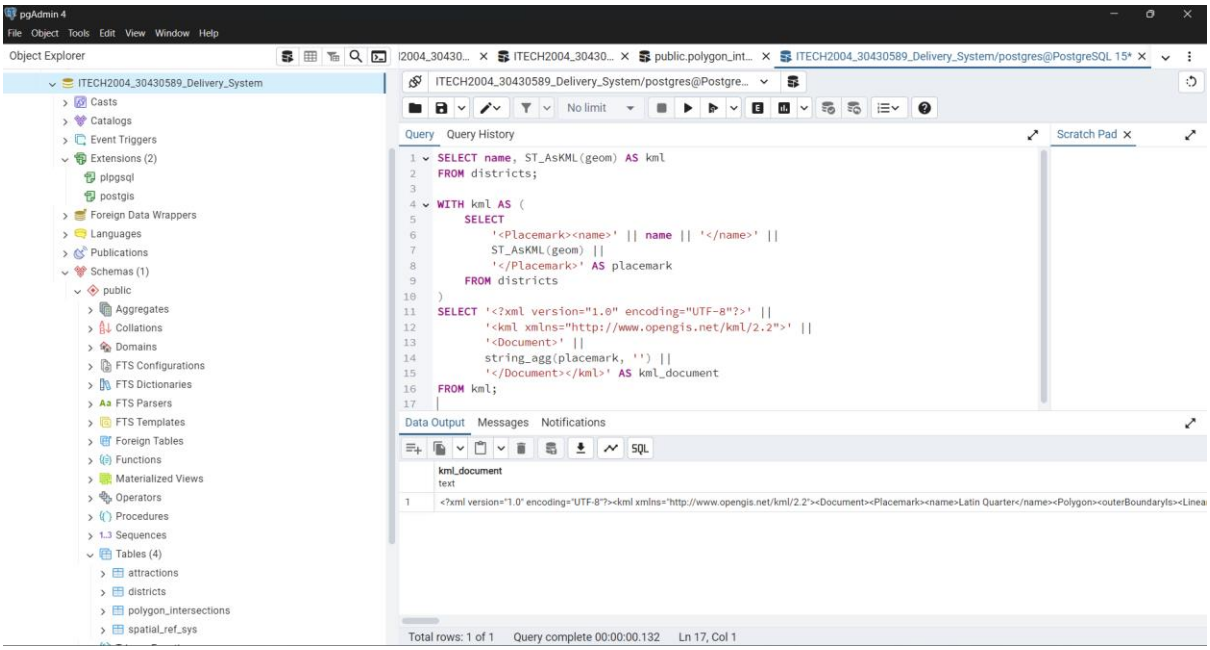
Task 5: Intersection Over Union

- **Displaying all polygons**

```
SELECT
   ST_AsKML(ST_Union(geom)) AS combined_kml
FROM districts;

WITH kml AS (
   SELECT
      '<Placemark><name>' || name || '</name>' ||
      ST_AsKML(geom) ||
      '</Placemark>' AS placemark
   FROM districts
)
SELECT '<?xml version="1.0" encoding="UTF-8"?>' ||
      '<kml xmlns="http://www.opengis.net/kml/2.2">' ||
      '<Document>' ||
      string_agg(placemark, '') ||
      '</Document></kml>' AS kml_document
FROM kml;
```

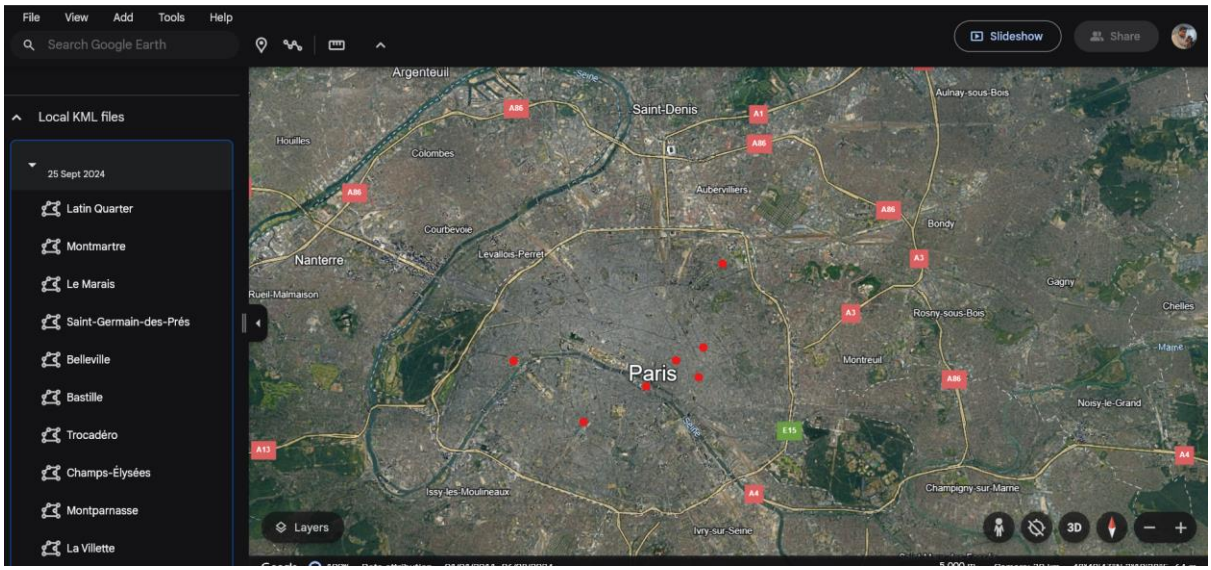
This SQL query converts polygon data from the districts table into a KML format for use in mapping tools like Google Earth. It generates placemarks for each district by wrapping the polygon geometries in KML-specific tags using the ST_AsKML function. These placemarks are then concatenated into a full KML document, with the necessary XML and KML headers. The result is a KML file that can be saved and opened in Google Earth to display the districts

visually. Overall, this query automates the process of transforming spatial data into a format compatible with geographic visualization tools.



<?xml version="1.0" encoding="UTF-8"?><kml xmlns="http://www.opengis.net/kml/2.2"><Document><Placemark><name>Latin Quarter</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3485,48.8534 2.3486,48.8535 2.3487,48.8536 2.3488,48.8537 2.3489,48.8538 2.349,48.8539 2.3491,48.854 2.3492,48.8541 2.3493,48.8542 2.3494,48.8543 2.3485,48.8534</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Montmartre</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.349,48.8536 2.3491,48.8537 2.3492,48.8538 2.3493,48.8539 2.3494,48.854 2.3495,48.8541 2.3496,48.8542 2.3497,48.8543 2.3498,48.8544 2.3499,48.8545 2.349,48.8536</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Le Marais</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.362,48.8615 2.3621,48.8616 2.3622,48.8617 2.3623,48.8618 2.3624,48.8619 2.3625,48.862 2.3626,48.8621 2.3627,48.8622 2.3628,48.8623 2.3629,48.8624 2.362,48.8615</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Saint-Germain-des-Prés</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3625,48.8617 2.3626,48.8618 2.3627,48.8619 2.3628,48.862 2.3629,48.8621 2.363,48.8622 2.3631,48.8623 2.3632,48.8624 2.3633,48.8625 2.3634,48.8626 2.3625,48.8617</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Belleville</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3745,48.865 2.3746,48.8651 2.3747,48.8652 2.3748,48.8653 2.3749,48.8654 2.375,48.8655 2.3751,48.8656 2.3752,48.8657 2.3753,48.8658 2.3754,48.8659 2.3745,48.865</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Bastille</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3725,48.856 2.3726,48.8561 2.3727,48.8562 2.3728,48.8563 2.3729,48.8564 2.373,48.8565 2.3731,48.8566 2.3732,48.8567 2.3733,48.8568 2.3734,48.8569 2.3725,48.856</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Trocadéro</name><Polygon><outerBoundaryIs><LinearRing><coordinate

s>2.2885,48.8611 2.2886,48.8612 2.2887,48.8613 2.2888,48.8614 2.2889,48.8615
2.289,48.8616 2.2891,48.8617 2.2892,48.8618 2.2893,48.8619 2.2894,48.862
2.2885,48.8611</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><
Placemark><name>Champs-
Élysées</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.289,48.8613
2.2891,48.8614 2.2892,48.8615 2.2893,48.8616 2.2894,48.8617 2.2895,48.8618
2.2896,48.8619 2.2897,48.862 2.2898,48.8621 2.2899,48.8622
2.289,48.8613</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Pl
acemark><name>Montparnasse</name><Polygon><outerBoundaryIs><LinearRing><coordi
nates>2.3205,48.8427 2.3206,48.8428 2.3207,48.8429 2.3208,48.843 2.3209,48.8431
2.321,48.8432 2.3211,48.8433 2.3212,48.8434 2.3213,48.8435 2.3214,48.8436
2.3205,48.8427</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><
Placemark><name>La
Villette</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3835,48.89
2.3836,48.8901 2.3837,48.8902 2.3838,48.8903 2.3839,48.8904 2.384,48.8905
2.3841,48.8906 2.3842,48.8907 2.3843,48.8908 2.3844,48.8909
2.3835,48.89</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark></D
ocument></kml>



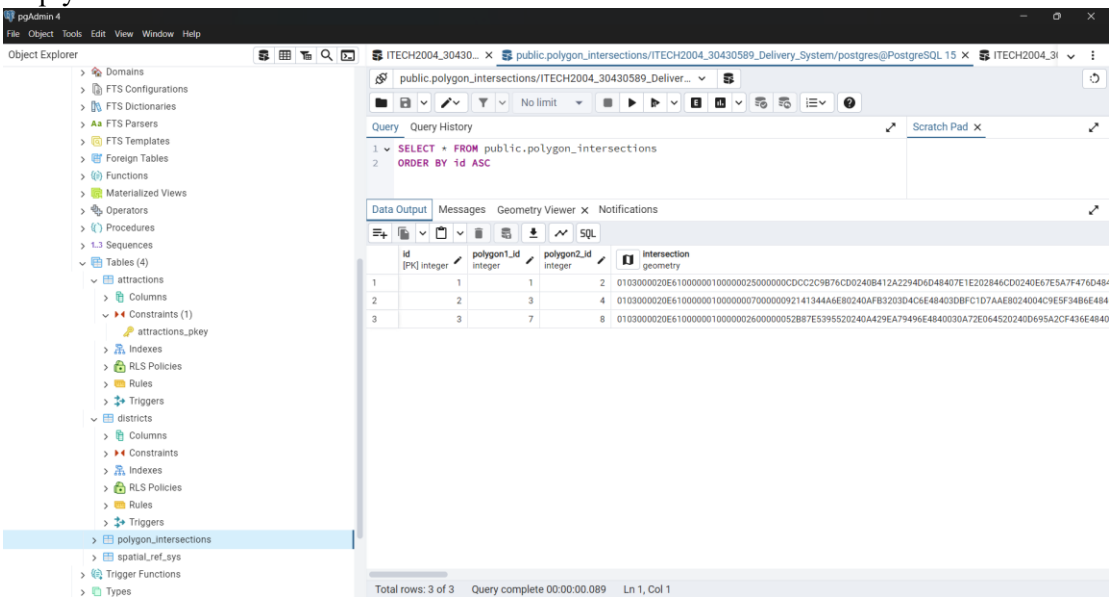
- **Displaying all intersections**

  - **Creating the table for polygon intersections**

    CREATE TABLE polygon_intersections (
      id SERIAL PRIMARY KEY,
      polygon1_id INT,
      polygon2_id INT,
      intersection GEOMETRY(Polygon, 4326),

```
    iou FLOAT
);

WITH polygon_pairs AS (
    SELECT
        p1.id AS polygon1_id,
        p2.id AS polygon2_id,
        ST_Intersection(ST_Buffer(p1.geom, 0.001), ST_Buffer(p2.geom, 0.001)) AS
intersection_geom,
        ST_Union(ST_Buffer(p1.geom, 0.001), ST_Buffer(p2.geom, 0.001)) AS
union_geom
    FROM districts p1
    JOIN districts p2 ON p1.id < p2.id
)
INSERT INTO polygon_intersections (polygon1_id, polygon2_id, intersection, iou)
SELECT
    polygon1_id,
    polygon2_id,
    intersection_geom,
    CASE
        WHEN ST_Area(union_geom) > 0
        THEN ST_Area(intersection_geom) / ST_Area(union_geom)
        ELSE 0
    END AS iou
FROM polygon_pairs
WHERE NOT ST_IsEmpty(intersection_geom);
```

This query creates a table called polygon_intersections to store the intersections of polygon pairs along with the Intersection over Union (IoU) value. It then calculates the geometric intersection and union of pairs of polygons from the districts table using a small buffer (0.001 units). The query inserts the results into the polygon_intersections table, including the polygon IDs, the intersection geometry, and the IoU, which is the ratio of the intersection area to the union area. Only non-empty intersections are inserted.



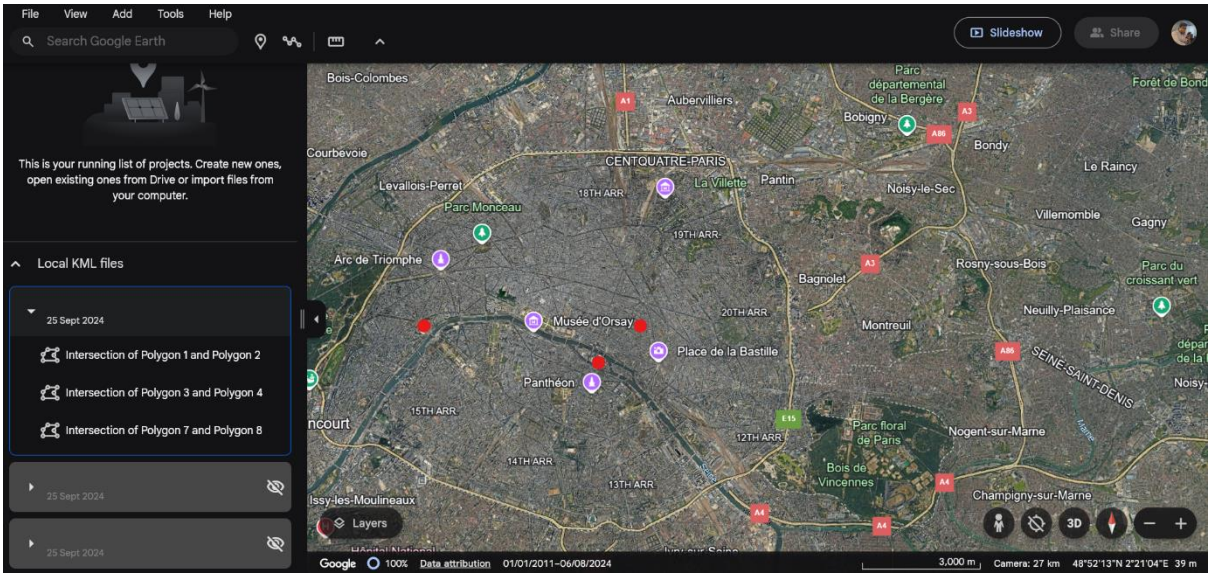
- **Generating the KML**

```
WITH intersection_kml AS (
   SELECT
      pi.id,
      'Intersection of Polygon ' || pi.polygon1_id || ' and Polygon ' || pi.polygon2_id AS
name,
      ST_AsKML(pi.intersection) AS kml
   FROM polygon_intersections pi
   WHERE NOT ST_IsEmpty(pi.intersection)
)
SELECT
   '<?xml version="1.0" encoding="UTF-8"?>' ||
   '<kml xmlns="http://www.opengis.net/kml/2.2">' ||
   '<Document>' ||
   string_agg('<Placemark><name>' || name || '</name>' || kml || '</Placemark>', '') ||
   '</Document></kml>' AS kml_document
FROM intersection_kml;
```

This query generates a KML document for the intersections of polygons. It first creates a temporary result set (intersection_kml) that contains the intersection IDs, names, and the KML representation of the intersection geometries. The outer query then formats this data into a full KML document, wrapping each intersection in a <Placemark> tag with its name and coordinates. The final result is a complete KML document, suitable for displaying the polygon intersections in Google Earth.


```
<?xml version="1.0" encoding="UTF-8"?><kml
xmlns="http://www.opengis.net/kml/2.2"><Document><Placemark><name>Intersect
ion of Polygon 1 and Polygon
2</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.35032387953
2516,48.853917316567646 2.35023146961231,48.85374442976699
2.350107106781198,48.853592893218824 2.350007106781174,48.8534928932188
2.349907106781198,48.853392893218825 2.349807106781174,48.8532928932188
2.349707106781198,48.853192893218825 2.349607106781174,48.8530928932188
2.349507106781198,48.852992893218826 2.349307106781174,48.8527928932188
2.349207106781174,48.8526928932188 2.349106724273064,48.85261051142343
2.349000000000006,48.8526 2.348804909677992,48.852619214719596
2.348617316567643,48.852676120467486 2.348444429766989,48.85276853038769
2.348292893218821,48.85289289321881 2.348168530387704,48.85304442976697
2.348076120467494,48.85321731656762 2.3480192147196,48.85340490967797
2.348,48.853599999999986 2.348019214719594,48.853795090322
2.348076120467482,48.85398268343235 2.348168530387687,48.854155570233004
2.348292893218801,48.85430710678117 2.348392893218801,48.854407106781174
2.348492893218828,48.8545071067812 2.348592893218801,48.854607106781174
2.348692893218828,48.8547071067812 2.348792893218801,48.85480710678117
2.348892893218828,48.8549071067812 2.348992893218801,48.85500710678117
2.349092893218828,48.8551071067812 2.349192893218801,48.85520710678117
```

2.349201622932149,48.85519837706782 2.350360840255984,48.85403915974401
2.350323879532516,48.853917316567646</coordinates></LinearRing></outerBoun
daryIs></Polygon></Placemark><Placemark><name>Intersection of Polygon 3 and
Polygon
4</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.36359837706
7839,48.861701622932166 2.363607106781187,48.861692893218816
2.363057106781195,48.861142893218826
2.361801622932161,48.862398377067834
2.361792893218813,48.862407106781184
2.362342893218801,48.862957106781174
2.363598377067839,48.861701622932166</coordinates></LinearRing></outerBoun
daryIs></Polygon></Placemark><Placemark><name>Intersection of Polygon 7 and
Polygon
8</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.29032387953
2516,48.861617316567646 2.29023146961231,48.86144442976699
2.290107106781198,48.861292893218824 2.290007106781174,48.8611928932188
2.289907106781198,48.861092893218824 2.289807106781174,48.8609928932188
2.289707106781198,48.860892893218825 2.289607106781174,48.8607928932188
2.289507106781198,48.860692893218825 2.289407106781199,48.86059289321883
2.289307106781173,48.8604928932188 2.289207106781173,48.8603928932188
2.289106724273063,48.86031051142343 2.289000000000006,48.8603
2.288804909677991,48.860319214719595
2.288617316567643,48.860376120467485 2.288444429766989,48.86046853038769
2.288292893218821,48.86059289321881 2.288168530387704,48.86074442976697
2.288076120467494,48.86091731656762 2.2880192147196,48.86110490967797
2.288,48.861299999999986 2.288019214719594,48.861495090322
2.288076120467482,48.86168268343235 2.288168530387687,48.861855570233004
2.288292893218801,48.86200710678117 2.288392893218801,48.862107106781174
2.288492893218828,48.8622071067812 2.288592893218801,48.862307106781174
2.288692893218828,48.8624071067812 2.288792893218801,48.86250710678117
2.288892893218828,48.8626071067812 2.288992893218801,48.86270710678117
2.289092893218828,48.8628071067812 2.289192893218801,48.86290710678117
2.289201622932148,48.86289837706782 2.290360840255985,48.86173915974401
2.290323879532516,48.861617316567646</coordinates></LinearRing></outerBoun
daryIs></Polygon></Placemark></Document></kml>



Task 6: Theory and Context

- **Three Additional Types of Functionalities**

– **Nearest Attraction Search by Radius**

```
CREATE OR REPLACE FUNCTION find_nearby_attractions(x_coordinate FLOAT,
y_coordinate FLOAT, radius FLOAT)
RETURNS TABLE(name VARCHAR, distance FLOAT) AS $$
BEGIN
   RETURN QUERY
   SELECT name, ST_Distance(geom, ST_SetSRID(ST_MakePoint(x_coordinate,
y_coordinate), 4326)) AS distance
   FROM attractions
   WHERE ST_DWithin(geom, ST_SetSRID(ST_MakePoint(x_coordinate,
y_coordinate), 4326), radius)
   ORDER BY distance ASC;
END $$ LANGUAGE plpgsql;
```

The function find_nearby_attractions accepts three inputs: the x_coordinate and y_coordinate of the user's location, and a radius to search within. The query uses ST_DWithin, a spatial function that checks if a point (the attraction's location) is within a certain distance from another point (the user's location).
The attractions are returned along with their calculated distances from the user's location, using the ST_Distance function to compute the actual distance. This function allows users to search for attractions that are located within a certain distance (radius) from a specified point (such as a user's location). This is useful for

tourists who want to know what attractions are nearby, or for a tour company creating custom itineraries based on location.

- **Finding District by Attraction**

```
CREATE OR REPLACE FUNCTION find_district_for_attraction(attraction_name
VARCHAR)
RETURNS TEXT AS $$
DECLARE
   district_name TEXT;
BEGIN
   SELECT d.name INTO district_name
   FROM districts d
   JOIN attractions a ON ST_Contains(d.geom, a.geom)
   WHERE a.name = attraction_name;

   RETURN district_name;
END $$ LANGUAGE plpgsql;
```

The find_district_for_attraction function accepts the attraction_name as an input, representing the name of an attraction. The function queries the districts table and uses the ST_Contains function, which checks if the geometry of a district (polygon) contains the geometry of the attraction (point). The name of the district is returned, allowing users or administrators to know which district is responsible for managing the attraction. This function helps identify which district a particular attraction belongs to. In the case of a tourist guide application or city management tool, it's important to know the administrative boundaries that contain specific points of interest.

- **Attractions by District Count**

```
CREATE OR REPLACE FUNCTION attractions_count_by_district()
RETURNS TABLE(district_name VARCHAR, attraction_count INT) AS $$
BEGIN
   RETURN QUERY
   SELECT d.name, COUNT(a.id) AS attraction_count
   FROM districts d
   LEFT JOIN attractions a ON ST_Contains(d.geom, a.geom)
   GROUP BY d.name;
END $$ LANGUAGE plpgsql;
```

The attractions_count_by_district function does not take any inputs and returns a table of district names and the count of attractions within each district. It uses a LEFT JOIN to join the districts and attractions tables based on whether the attraction's point is contained within the district’s polygon, determined using ST_Contains. The query groups the results by district name and counts the number of attractions for each district. This function counts how many attractions are located within each district. It is particularly useful for city planners or tourism management teams who need to

assess which districts are most populated with attractions, for reasons like infrastructure planning or promotional efforts.

- **Implementing Functions without**

  - **Spatial Extensions**

    1. **Nearest Attraction Search by Radius**

       ```
       SELECT name,
          SQRT(POWER((x_coordinate - latitude), 2) + POWER((y_coordinate - longitude), 2)) AS distance
       FROM attractions
       WHERE SQRT(POWER((x_coordinate - latitude), 2) +
       POWER((y_coordinate - longitude), 2)) < radius
       ORDER BY distance;
       ```

       Without spatial extensions, we can calculate the distance between two points manually using the Euclidean distance formula or, for larger areas, the Haversine formula, which accounts for the curvature of the Earth. This query manually calculates the Euclidean distance between two points using basic arithmetic. The formula SQRT(POWER((x_coordinate - latitude), 2) + POWER((y_coordinate - longitude), 2)) calculates the straight-line distance between the given point and the attraction. The WHERE clause filters attractions that are within the specified radius and the ORDER BY clause orders the results by the distance. This is less accurate than using ST_DWithin and ST_Distance for large areas but works for smaller geographical regions.

    2. **Finding District by Attraction**

       ```
       SELECT d.name
       FROM districts d, attractions a
       WHERE a.name = 'Eiffel Tower'
       AND a.latitude BETWEEN d.min_latitude AND d.max_latitude
       AND a.longitude BETWEEN d.min_longitude AND d.max_longitude;
       ```

       Without spatial extensions, the relationship between a district and its attractions can be approximated using pre-calculated bounding boxes for each district. A bounding box is defined by the minimum and maximum latitudes and longitudes. This query manually checks if the latitude and longitude of the attraction fall within the minimum and maximum latitude/longitude of a district. It approximates the use of ST_Contains by treating the district as a rectangular box (bounding box) rather than a more accurate polygon. This solution is less precise since it cannot account for the actual shape of a district, only its bounding rectangle, but it is still functional for rough containment checks.

3. **Attractions by District Count**

```
SELECT d.name, COUNT(a.id) AS attraction_count
FROM districts d
LEFT JOIN attractions a
ON a.latitude BETWEEN d.min_latitude AND d.max_latitude
AND a.longitude BETWEEN d.min_longitude AND d.max_longitude
GROUP BY d.name;
```

A similar bounding box check is performed but the results are grouped by district, to count the number of attractions within each district. The query checks if each attraction's latitude and longitude fall within the bounding box of a district. The GROUP BY d.name clause ensures that the results are grouped by district, and COUNT(a.id) counts how many attractions fall within each district. This is an approximate method compared to using actual polygons but achieves a similar result.

- **Procedural Extensions**

1. **Nearest Attraction Search by Radius**

```
SELECT name,
    ST_Distance(geom, ST_SetSRID(ST_MakePoint(x_coordinate,
y_coordinate), 4326)) AS distance
FROM attractions
WHERE ST_DWithin(geom, ST_SetSRID(ST_MakePoint(x_coordinate,
y_coordinate), 4326), radius)
ORDER BY distance ASC;
```

In this query, the user provides x_coordinate, y_coordinate, and radius as inputs. The query makes use of the ST_DWithin function to find points (attractions) within a specified radius of a location. It then calculates the actual distance between the specified location and each attraction using the ST_Distance function. The results are ordered by distance, making it possible to easily identify which attractions are nearest to the provided point. While this approach does not encapsulate the logic in a reusable function, it offers the same functionality in one-off query executions.

2. **Finding District by Attraction**

```
SELECT d.name
FROM districts d
JOIN attractions a ON ST_Contains(d.geom, a.geom)
WHERE a.name = 'attraction_name';
```

In this case, the query uses the ST_Contains function to determine if the geometry of a district contains the geometry of an attraction (a point). By joining the districts and attractions tables and applying this spatial condition, the query returns the district that encompasses the specified attraction. Users

can replace 'attraction_name' with the actual name of the attraction they are interested in. This SQL query provides the same result as a procedural function but must be executed separately each time a district lookup is required.

3. **Attractions by District Count**

```
SELECT d.name AS district_name,
      COUNT(a.id) AS attraction_count
FROM districts d
LEFT JOIN attractions a ON ST_Contains(d.geom, a.geom)
GROUP BY d.name;
```

This query performs a LEFT JOIN between the districts and attractions tables, checking whether each attraction point is contained within a district polygon using the ST_Contains function. The query then groups the results by district name and counts how many attractions exist within each district. The result is a list of districts along with the number of attractions they contain. This approach provides the same insight as a procedural function but requires executing the query each time updated attraction counts are needed.

Appendix

1. **Setting up the Database**

```
/* This script file creates the following tables:       */
/* districts, attractions                               */
/* and loads the default data rows                      */

START TRANSACTION;

-- Drop existing tables if they exist
DROP TABLE IF EXISTS districts;
DROP TABLE IF EXISTS attractions;

-- Create the 'districts' table
CREATE TABLE districts (
   id SERIAL PRIMARY KEY,
   name VARCHAR(100),
   geom GEOMETRY(POLYGON, 4326)
);

-- Create the 'attractions' table
CREATE TABLE attractions (
```

```
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    geom GEOMETRY(POINT, 4326)
);

COMMIT;

/* Loading data rows */
START TRANSACTION;

/* DISTRICTS rows */
/* DISTRICTS rows */
INSERT INTO districts (name, geom)
VALUES
('Latin Quarter', ST_GeomFromText('POLYGON((2.3485 48.8534, 2.3486 48.8535, 2.3487
48.8536, 2.3488 48.8537, 2.3489 48.8538, 2.3490 48.8539, 2.3491 48.8540, 2.3492 48.8541,
2.3493 48.8542, 2.3494 48.8543, 2.3485 48.8534))', 4326)),
('Montmartre', ST_GeomFromText('POLYGON((2.3490 48.8536, 2.3491 48.8537, 2.3492
48.8538, 2.3493 48.8539, 2.3494 48.8540, 2.3495 48.8541, 2.3496 48.8542, 2.3497 48.8543,
2.3498 48.8544, 2.3499 48.8545, 2.3490 48.8536))', 4326)),
('Le Marais', ST_GeomFromText('POLYGON((2.3620 48.8615, 2.3621 48.8616, 2.3622
48.8617, 2.3623 48.8618, 2.3624 48.8619, 2.3625 48.8620, 2.3626 48.8621, 2.3627 48.8622,
2.3628 48.8623, 2.3629 48.8624, 2.3620 48.8615))', 4326)),
('Saint-Germain-des-Prés', ST_GeomFromText('POLYGON((2.3625 48.8617, 2.3626
48.8618, 2.3627 48.8619, 2.3628 48.8620, 2.3629 48.8621, 2.3630 48.8622, 2.3631 48.8623,
2.3632 48.8624, 2.3633 48.8625, 2.3634 48.8626, 2.3625 48.8617))', 4326)),
('Belleville', ST_GeomFromText('POLYGON((2.3745 48.8650, 2.3746 48.8651, 2.3747
48.8652, 2.3748 48.8653, 2.3749 48.8654, 2.3750 48.8655, 2.3751 48.8656, 2.3752 48.8657,
2.3753 48.8658, 2.3754 48.8659, 2.3745 48.8650))', 4326)),
('Bastille', ST_GeomFromText('POLYGON((2.3725 48.8560, 2.3726 48.8561, 2.3727
48.8562, 2.3728 48.8563, 2.3729 48.8564, 2.3730 48.8565, 2.3731 48.8566, 2.3732 48.8567,
2.3733 48.8568, 2.3734 48.8569, 2.3725 48.8560))', 4326)),
('Trocadéro', ST_GeomFromText('POLYGON((2.2885 48.8611, 2.2886 48.8612, 2.2887
48.8613, 2.2888 48.8614, 2.2889 48.8615, 2.2890 48.8616, 2.2891 48.8617, 2.2892 48.8618,
2.2893 48.8619, 2.2894 48.8620, 2.2885 48.8611))', 4326)),
('Champs-Élysées', ST_GeomFromText('POLYGON((2.2890 48.8613, 2.2891 48.8614,
2.2892 48.8615, 2.2893 48.8616, 2.2894 48.8617, 2.2895 48.8618, 2.2896 48.8619, 2.2897
48.8620, 2.2898 48.8621, 2.2899 48.8622, 2.2890 48.8613))', 4326)),
('Montparnasse', ST_GeomFromText('POLYGON((2.3205 48.8427, 2.3206 48.8428, 2.3207
48.8429, 2.3208 48.8430, 2.3209 48.8431, 2.3210 48.8432, 2.3211 48.8433, 2.3212 48.8434,
2.3213 48.8435, 2.3214 48.8436, 2.3205 48.8427))', 4326)),
('La Villette', ST_GeomFromText('POLYGON((2.3835 48.8900, 2.3836 48.8901, 2.3837
48.8902, 2.3838 48.8903, 2.3839 48.8904, 2.3840 48.8905, 2.3841 48.8906, 2.3842 48.8907,
2.3843 48.8908, 2.3844 48.8909, 2.3835 48.8900))', 4326));

/* ATTRACTIONS rows */
INSERT INTO attractions (name, geom)
VALUES
('Eiffel Tower', ST_SetSRID(ST_MakePoint(2.2945, 48.8584), 4326));
INSERT INTO attractions (name, geom)
VALUES
```

('Louvre Museum', ST_SetSRID(ST_MakePoint(2.3376, 48.8606), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Notre-Dame Cathedral', ST_SetSRID(ST_MakePoint(2.3489, 48.8538), 4326)); -- Modified to be within Latin Quarter
INSERT INTO attractions (name, geom)
VALUES
('Arc de Triomphe', ST_SetSRID(ST_MakePoint(2.2950, 48.8738), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Sacré-Cœur Basilica', ST_SetSRID(ST_MakePoint(2.3494, 48.8540), 4326)); -- Modified to be within Montmartre
INSERT INTO attractions (name, geom)
VALUES
('Musée d''Orsay', ST_SetSRID(ST_MakePoint(2.3266, 48.8600), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Palais Garnier', ST_SetSRID(ST_MakePoint(2.3318, 48.8704), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Luxembourg Gardens', ST_SetSRID(ST_MakePoint(2.3629, 48.8620), 4326)); -- Modified to be within Saint-Germain-des-Prés
INSERT INTO attractions (name, geom)
VALUES
('Place de la Concorde', ST_SetSRID(ST_MakePoint(2.3212, 48.8656), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Centre Pompidou', ST_SetSRID(ST_MakePoint(2.3522, 48.8606), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Pont Alexandre III', ST_SetSRID(ST_MakePoint(2.2889, 48.8615), 4326)); -- Modified to be within Trocadéro
INSERT INTO attractions (name, geom)
VALUES
('Pantheon', ST_SetSRID(ST_MakePoint(2.3461, 48.8462), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Place des Vosges', ST_SetSRID(ST_MakePoint(2.3662, 48.8554), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Tuileries Garden', ST_SetSRID(ST_MakePoint(2.3270, 48.8635), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rodin Museum', ST_SetSRID(ST_MakePoint(2.3156, 48.8556), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Champs-Élysées', ST_SetSRID(ST_MakePoint(2.2894, 48.8617), 4326)); -- Modified to be within Champs-Élysées district
INSERT INTO attractions (name, geom)
VALUES
('Moulin Rouge', ST_SetSRID(ST_MakePoint(2.3326, 48.8841), 4326));

```
INSERT INTO attractions (name, geom)
VALUES
('Les Invalides', ST_SetSRID(ST_MakePoint(2.3126, 48.8566), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Palace of Versailles', ST_SetSRID(ST_MakePoint(2.1203, 48.8049), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Sainte-Chapelle', ST_SetSRID(ST_MakePoint(2.3450, 48.8554), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Parc Monceau', ST_SetSRID(ST_MakePoint(2.3090, 48.8793), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Montparnasse Tower', ST_SetSRID(ST_MakePoint(2.3213, 48.8422), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Père Lachaise Cemetery', ST_SetSRID(ST_MakePoint(2.3933, 48.8614), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Villette Park', ST_SetSRID(ST_MakePoint(2.3911, 48.8896), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Place de la Bastille', ST_SetSRID(ST_MakePoint(2.3690, 48.8530), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Palais Royal', ST_SetSRID(ST_MakePoint(2.3364, 48.8635), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Pont Neuf', ST_SetSRID(ST_MakePoint(2.3429, 48.8585), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Défense Grande Arche', ST_SetSRID(ST_MakePoint(2.2380, 48.8919), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Jardin des Plantes', ST_SetSRID(ST_MakePoint(2.3606, 48.8447), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Galeries Lafayette', ST_SetSRID(ST_MakePoint(2.3323, 48.8738), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée de l''Orangerie', ST_SetSRID(ST_MakePoint(2.3224, 48.8629), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Canal Saint-Martin', ST_SetSRID(ST_MakePoint(2.3646, 48.8683), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Conciergerie', ST_SetSRID(ST_MakePoint(2.3443, 48.8555), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Église Saint-Sulpice', ST_SetSRID(ST_MakePoint(2.3342, 48.8515), 4326));
```

```
INSERT INTO attractions (name, geom)
VALUES
('Musée Jacquemart-André', ST_SetSRID(ST_MakePoint(2.3097, 48.8762), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Institut de France', ST_SetSRID(ST_MakePoint(2.3365, 48.8573), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée Picasso', ST_SetSRID(ST_MakePoint(2.3622, 48.8596), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Aquarium de Paris', ST_SetSRID(ST_MakePoint(2.2887, 48.8616), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée Marmottan Monet', ST_SetSRID(ST_MakePoint(2.2685, 48.8583), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Conciergerie', ST_SetSRID(ST_MakePoint(2.3444, 48.8555), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Bourse de Commerce', ST_SetSRID(ST_MakePoint(2.3416, 48.8631), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Île de la Cité', ST_SetSRID(ST_MakePoint(2.3455, 48.8556), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Le Marais', ST_SetSRID(ST_MakePoint(2.3620, 48.8605), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Flame of Liberty', ST_SetSRID(ST_MakePoint(2.3014, 48.8641), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Maison de Victor Hugo', ST_SetSRID(ST_MakePoint(2.3663, 48.8545), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Le Petit Palais', ST_SetSRID(ST_MakePoint(2.3130, 48.8660), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Paris Catacombs', ST_SetSRID(ST_MakePoint(2.3321, 48.8339), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Saint-Germain-des-Prés', ST_SetSRID(ST_MakePoint(2.3333, 48.8549), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Boulevard Saint-Germain', ST_SetSRID(ST_MakePoint(2.3428, 48.8532), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Grand Palais', ST_SetSRID(ST_MakePoint(2.3126, 48.8662), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée Carnavalet', ST_SetSRID(ST_MakePoint(2.3621, 48.8579), 4326));
```

```
INSERT INTO attractions (name, geom)
VALUES
('Musée National d''Art Moderne', ST_SetSRID(ST_MakePoint(2.3522, 48.8606), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Parc des Buttes-Chaumont', ST_SetSRID(ST_MakePoint(2.3827, 48.8801), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue de Rivoli', ST_SetSRID(ST_MakePoint(2.3427, 48.8592), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Bois de Boulogne', ST_SetSRID(ST_MakePoint(2.2449, 48.8687), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Grande Mosquée de Paris', ST_SetSRID(ST_MakePoint(2.3551, 48.8416), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue Mouffetard', ST_SetSRID(ST_MakePoint(2.3492, 48.8414), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Paris Zoological Park', ST_SetSRID(ST_MakePoint(2.4164, 48.8345), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Parc André Citroën', ST_SetSRID(ST_MakePoint(2.2766, 48.8402), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue Montorgueil', ST_SetSRID(ST_MakePoint(2.3481, 48.8657), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Marché aux Fleurs Reine Elizabeth II', ST_SetSRID(ST_MakePoint(2.3470, 48.8563),
4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée des Arts Décoratifs', ST_SetSRID(ST_MakePoint(2.3349, 48.8634), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Saint-Jacques Tower', ST_SetSRID(ST_MakePoint(2.3498, 48.8583), 4326));
INSERT INTO attractions (name, geom)
VALUES
('National Museum of Natural History', ST_SetSRID(ST_MakePoint(2.3594, 48.8431),
4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue des Rosiers', ST_SetSRID(ST_MakePoint(2.3616, 48.8577), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Église de la Madeleine', ST_SetSRID(ST_MakePoint(2.3242, 48.8699), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Géode', ST_SetSRID(ST_MakePoint(2.3884, 48.8906), 4326));
INSERT INTO attractions (name, geom)
```

```
VALUES
('Forum des Halles', ST_SetSRID(ST_MakePoint(2.3470, 48.8617), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue de la Paix', ST_SetSRID(ST_MakePoint(2.3304, 48.8679), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Parc Floral de Paris', ST_SetSRID(ST_MakePoint(2.4341, 48.8369), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Pont des Arts', ST_SetSRID(ST_MakePoint(2.3371, 48.8584), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Colonne Vendôme', ST_SetSRID(ST_MakePoint(2.3282, 48.8676), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue du Faubourg Saint-Honoré', ST_SetSRID(ST_MakePoint(2.3221, 48.8690), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée du Quai Branly', ST_SetSRID(ST_MakePoint(2.2974, 48.8603), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Palais de Tokyo', ST_SetSRID(ST_MakePoint(2.2960, 48.8650), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Grande Galerie de l''Évolution', ST_SetSRID(ST_MakePoint(2.3561, 48.8435), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Place Vendôme', ST_SetSRID(ST_MakePoint(2.3284, 48.8674), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue des Martyrs', ST_SetSRID(ST_MakePoint(2.3426, 48.8791), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Montparnasse Cemetery', ST_SetSRID(ST_MakePoint(2.3267, 48.8381), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Le Bon Marché', ST_SetSRID(ST_MakePoint(2.3244, 48.8501), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Marché d''Aligre', ST_SetSRID(ST_MakePoint(2.3802, 48.8495), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Musée Guimet', ST_SetSRID(ST_MakePoint(2.2937, 48.8641), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Avenue Foch', ST_SetSRID(ST_MakePoint(2.2829, 48.8728), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue du Bac', ST_SetSRID(ST_MakePoint(2.3267, 48.8543), 4326));
INSERT INTO attractions (name, geom)
```

```
VALUES
('Pavillon de l''Arsenal', ST_SetSRID(ST_MakePoint(2.3666, 48.8539), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Parc Montsouris', ST_SetSRID(ST_MakePoint(2.3405, 48.8217), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Pont de Bir-Hakeim', ST_SetSRID(ST_MakePoint(2.2891, 48.8551), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Château de Vincennes', ST_SetSRID(ST_MakePoint(2.4364, 48.8414), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Palais de Chaillot', ST_SetSRID(ST_MakePoint(2.2882, 48.8625), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Gare Saint-Lazare', ST_SetSRID(ST_MakePoint(2.3276, 48.8760), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Gare de Lyon', ST_SetSRID(ST_MakePoint(2.3730, 48.8443), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Place d''Italie', ST_SetSRID(ST_MakePoint(2.3556, 48.8327), 4326));
INSERT INTO attractions (name, geom)
VALUES
('École Militaire', ST_SetSRID(ST_MakePoint(2.3030, 48.8537), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Rue Oberkampf', ST_SetSRID(ST_MakePoint(2.3766, 48.8646), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Square du Vert-Galant', ST_SetSRID(ST_MakePoint(2.3400, 48.8572), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Gare du Nord', ST_SetSRID(ST_MakePoint(2.3553, 48.8809), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Église Saint-Eustache', ST_SetSRID(ST_MakePoint(2.3450, 48.8626), 4326));
INSERT INTO attractions (name, geom)
VALUES
('La Sorbonne University', ST_SetSRID(ST_MakePoint(2.3435, 48.8487), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Galerie Vivienne', ST_SetSRID(ST_MakePoint(2.3416, 48.8669), 4326));
INSERT INTO attractions (name, geom)
VALUES
('Canal de l''Ourcq', ST_SetSRID(ST_MakePoint(2.3906, 48.8922), 4326));

COMMIT;
```

2. **SQL Queries for Spatial Analysis**

- **Example of INNER JOIN**

```
SELECT a.name, d.name
FROM attractions a
INNER JOIN districts d ON ST_DWithin(a.geom, d.geom, 0.001);
```

- **Example of OUTER JOIN**

```
SELECT a.name, d.name
FROM attractions a
LEFT OUTER JOIN districts d
ON ST_DWithin(a.geom, d.geom, 0.001);
```

- **Example of GROUP BY and HAVING**

```
SELECT d.name, COUNT(a.id) AS attraction_count
FROM districts d
LEFT JOIN attractions a ON ST_DWithin(a.geom, d.geom, 0.005)
GROUP BY d.name
HAVING COUNT(a.id) > 5;
```

- **Example of SELECT**

```
SELECT a.name, ST_AsText(a.geom) AS geom
FROM attractions a
WHERE id IN (
   SELECT a.id
   FROM attractions a
   JOIN districts d ON ST_DWithin(a.geom, d.geom, 0.005)
   WHERE d.name = 'Latin Quarter'
);
```

- **Finding the Nearest Polygon to a Point**

```
SELECT name, ST_Distance(geom, ST_SetSRID(ST_MakePoint(2.2945, 48.8584),
4326)) AS distance
FROM districts
ORDER BY distance
LIMIT 1;
```

3. **Extended SQL Functionality**

- **Creating a Stored Procedure**

```
CREATE OR REPLACE PROCEDURE find_district_for_point(x_coordinate FLOAT,
y_coordinate FLOAT)
LANGUAGE plpgsql
AS $$
DECLARE
   district_name TEXT;
BEGIN
   SELECT name INTO district_name
   FROM districts
   WHERE ST_Contains(geom, ST_SetSRID(ST_MakePoint(x_coordinate,
y_coordinate), 4326));
   IF district_name IS NULL THEN
      RAISE NOTICE 'No district found for the given point';
   ELSE
      RAISE NOTICE 'The point is within the district: %', district_name;
   END IF;
END;
$$;
```

- **Creating a Trigger**

```
CREATE OR REPLACE FUNCTION check_point_within_bounding_box()
RETURNS TRIGGER AS $$
DECLARE
   bounding_box GEOMETRY := ST_SetSRID(
      ST_MakePolygon(
         ST_GeomFromText('LINESTRING(2.2900 48.8500, 2.2900 48.8600, 2.3000
48.8600, 2.3000 48.8500, 2.2900 48.8500)')
      ), 4326
   );
   point_inside BOOLEAN;
BEGIN
   point_inside := ST_Contains(bounding_box, NEW.geom);
   IF point_inside THEN
      RAISE NOTICE 'The point is within the bounding box';
   ELSE
      RAISE NOTICE 'The point is outside the bounding box';
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_insert_check_bounding_box
AFTER INSERT ON attractions
FOR EACH ROW
EXECUTE FUNCTION check_point_within_bounding_box();
```

4. **KML Queries**

- **Generating KML for Districts**

```
SELECT
  ST_AsKML(ST_Union(geom)) AS combined_kml
FROM districts;

WITH kml AS (
  SELECT
    '<Placemark><name>' || name || '</name>' ||
    ST_AsKML(geom) ||
    '</Placemark>' AS placemark
  FROM districts
)
SELECT '<?xml version="1.0" encoding="UTF-8"?>' ||
     '<kml xmlns="http://www.opengis.net/kml/2.2">' ||
     '<Document>' ||
     string_agg(placemark, '') ||
     '</Document></kml>' AS kml_document
FROM kml;
```

- **Creating Table for Polygon Intersections**

```
CREATE TABLE polygon_intersections (
  id SERIAL PRIMARY KEY,
  polygon1_id INT,
  polygon2_id INT,
  intersection GEOMETRY(POLYGON, 4326),
  iou FLOAT
);

WITH polygon_pairs AS (
  SELECT p1.id AS polygon1_id, p2.id AS polygon2_id,
       ST_Intersection(ST_Buffer(p1.geom, 0.001), ST_Buffer(p2.geom, 0.001)) AS
intersection_geom,
       ST_Union(ST_Buffer(p1.geom, 0.001), ST_Buffer(p2.geom, 0.001)) AS
union_geom
  FROM districts p1
  JOIN districts p2 ON p1.id < p2.id
)
INSERT INTO polygon_intersections (polygon1_id, polygon2_id, intersection, iou)
SELECT polygon1_id, polygon2_id, intersection_geom,
     CASE WHEN ST_Area(union_geom) > 0 THEN ST_Area(intersection_geom) /
ST_Area(union_geom) ELSE 0 END AS iou
FROM polygon_pairs
WHERE NOT ST_IsEmpty(intersection_geom);
```

- **Generating KML for Polygon Intersections**

```
WITH intersection_kml AS (
   SELECT
      pi.id,
      'Intersection of Polygon ' || pi.polygon1_id || ' and Polygon ' || pi.polygon2_id AS
name,
      ST_AsKML(pi.intersection) AS kml
   FROM polygon_intersections pi
   WHERE NOT ST_IsEmpty(pi.intersection)
)
SELECT
   '<?xml version="1.0" encoding="UTF-8"?>' ||
   '<kml xmlns="http://www.opengis.net/kml/2.2">' ||
   '<Document>' ||
   string_agg('<Placemark><name>' || name || '</name>' || kml || '</Placemark>', '') ||
   '</Document></kml>' AS kml_document
FROM intersection_kml;
```

- **KML for Districts**

```
<?xml version="1.0" encoding="UTF-8"?><kml
xmlns="http://www.opengis.net/kml/2.2"><Document><Placemark><name>Latin
Quarter</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3485,48.
8534 2.3486,48.8535 2.3487,48.8536 2.3488,48.8537 2.3489,48.8538 2.349,48.8539
2.3491,48.854 2.3492,48.8541 2.3493,48.8542 2.3494,48.8543
2.3485,48.8534</coordinates></LinearRing></outerBoundaryIs></Polygon></Placem
ark><Placemark><name>Montmartre</name><Polygon><outerBoundaryIs><Linear
Ring><coordinates>2.349,48.8536 2.3491,48.8537 2.3492,48.8538 2.3493,48.8539
2.3494,48.854 2.3495,48.8541 2.3496,48.8542 2.3497,48.8543 2.3498,48.8544
2.3499,48.8545
2.349,48.8536</coordinates></LinearRing></outerBoundaryIs></Polygon></Placema
rk><Placemark><name>Le
Marais</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.362,48.86
15 2.3621,48.8616 2.3622,48.8617 2.3623,48.8618 2.3624,48.8619 2.3625,48.862
2.3626,48.8621 2.3627,48.8622 2.3628,48.8623 2.3629,48.8624
2.362,48.8615</coordinates></LinearRing></outerBoundaryIs></Polygon></Placema
rk><Placemark><name>Saint-Germain-des-
Prés</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3625,48.861
7 2.3626,48.8618 2.3627,48.8619 2.3628,48.862 2.3629,48.8621 2.363,48.8622
2.3631,48.8623 2.3632,48.8624 2.3633,48.8625 2.3634,48.8626
2.3625,48.8617</coordinates></LinearRing></outerBoundaryIs></Polygon></Placem
ark><Placemark><name>Belleville</name><Polygon><outerBoundaryIs><LinearRin
g><coordinates>2.3745,48.865 2.3746,48.8651 2.3747,48.8652 2.3748,48.8653
2.3749,48.8654 2.375,48.8655 2.3751,48.8656 2.3752,48.8657 2.3753,48.8658
2.3754,48.8659
2.3745,48.865</coordinates></LinearRing></outerBoundaryIs></Polygon></Placema
rk><Placemark><name>Bastille</name><Polygon><outerBoundaryIs><LinearRing>
<coordinates>2.3725,48.856 2.3726,48.8561 2.3727,48.8562 2.3728,48.8563
```

2.3729,48.8564 2.373,48.8565 2.3731,48.8566 2.3732,48.8567 2.3733,48.8568
2.3734,48.8569
2.3725,48.856</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Trocadéro</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.2885,48.8611 2.2886,48.8612 2.2887,48.8613 2.2888,48.8614
2.2889,48.8615 2.289,48.8616 2.2891,48.8617 2.2892,48.8618 2.2893,48.8619
2.2894,48.862
2.2885,48.8611</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Champs-
Élysées</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.289,48.8613 2.2891,48.8614 2.2892,48.8615 2.2893,48.8616 2.2894,48.8617 2.2895,48.8618
2.2896,48.8619 2.2897,48.862 2.2898,48.8621 2.2899,48.8622
2.289,48.8613</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Montparnasse</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3205,48.8427 2.3206,48.8428 2.3207,48.8429 2.3208,48.843
2.3209,48.8431 2.321,48.8432 2.3211,48.8433 2.3212,48.8434 2.3213,48.8435
2.3214,48.8436
2.3205,48.8427</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>La
Villette</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.3835,48.89 2.3836,48.8901 2.3837,48.8902 2.3838,48.8903 2.3839,48.8904 2.384,48.8905
2.3841,48.8906 2.3842,48.8907 2.3843,48.8908 2.3844,48.8909
2.3835,48.89</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark></Document></kml>

- **KML for Polygon Intersections**

<?xml version="1.0" encoding="UTF-8"?><kml
xmlns="http://www.opengis.net/kml/2.2"><Document><Placemark><name>Intersection of Polygon 1 and Polygon
2</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.350323879532516,48.853917316567646 2.35023146961231,48.85374442976699
2.350107106781198,48.853592893218824 2.350007106781174,48.8534928932188
2.349907106781198,48.853392893218825 2.349807106781174,48.8532928932188
2.349707106781198,48.853192893218825 2.349607106781174,48.8530928932188
2.349507106781198,48.852992893218826 2.349307106781174,48.8527928932188
2.349207106781174,48.8526928932188 2.349106724273064,48.85261051142343
2.349000000000006,48.8526 2.348804909677992,48.852619214719596
2.348617316567643,48.852676120467486 2.348444429766989,48.85276853038769
2.348292893218821,48.85289289321881 2.348168530387704,48.85304442976697
2.348076120467494,48.85321731656762 2.3480192147196,48.85340490967797
2.348,48.853599999999986 2.348019214719594,48.853795090322
2.348076120467482,48.85398268343235 2.348168530387687,48.854155570233004
2.348292893218801,48.85430710678117 2.348392893218801,48.854407106781174
2.348492893218828,48.8545071067812 2.348592893218801,48.854607106781174
2.348692893218828,48.8547071067812 2.348792893218801,48.85480710678117
2.348892893218828,48.8549071067812 2.348992893218801,48.85500710678117
2.349092893218828,48.8551071067812 2.349192893218801,48.85520710678117
2.349201622932149,48.85519837706782 2.350360840255984,48.85403915974401

2.350323879532516,48.853917316567646</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Intersection of Polygon 3 and Polygon
4</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.363598377067839,48.861701622932166 2.363607106781187,48.861692893218816
2.363057106781195,48.861142893218826
2.361801622932161,48.862398377067834
2.361792893218813,48.862407106781184
2.362342893218801,48.862957106781174
2.363598377067839,48.861701622932166</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark><Placemark><name>Intersection of Polygon 7 and Polygon
8</name><Polygon><outerBoundaryIs><LinearRing><coordinates>2.290323879532516,48.861617316567646 2.29023146961231,48.86144442976699
2.290107106781198,48.861292893218824 2.290007106781174,48.8611928932188
2.289907106781198,48.861092893218824 2.289807106781174,48.8609928932188
2.289707106781198,48.860892893218825 2.289607106781174,48.8607928932188
2.289507106781198,48.860692893218825 2.289407106781199,48.86059289321883
2.289307106781173,48.8604928932188 2.289207106781173,48.8603928932188
2.289106724273063,48.86031051142343 2.289000000000006,48.8603
2.288804909677991,48.860319214719595
2.288617316567643,48.860376120467485 2.288444429766989,48.86046853038769
2.288292893218821,48.86059289321881 2.288168530387704,48.86074442976697
2.288076120467494,48.86091731656762 2.2880192147196,48.86110490967797
2.288,48.861299999999986 2.288019214719594,48.861495090322
2.288076120467482,48.86168268343235 2.288168530387687,48.861855570233004
2.288292893218801,48.86200710678117 2.288392893218801,48.862107106781174
2.288492893218828,48.8622071067812 2.288592893218801,48.862307106781174
2.288692893218828,48.8624071067812 2.288792893218801,48.86250710678117
2.288892893218828,48.8626071067812 2.288992893218801,48.86270710678117
2.289092893218828,48.8628071067812 2.289192893218801,48.86290710678117
2.289201622932148,48.86289837706782 2.290360840255985,48.86173915974401
2.290323879532516,48.861617316567646</coordinates></LinearRing></outerBoundaryIs></Polygon></Placemark></Document></kml>